



OPERATING SYSTEM

OS-LAB Project

ABSTRACT

MaqOS (Maqsad Operating System) is a lightweight educational operating system developed by M. Taha Rehan, M. Abdullah, and Junaid Haider to demonstrate core OS concepts.

Supervisor:

Mr. Mozeb Ahmed Khan

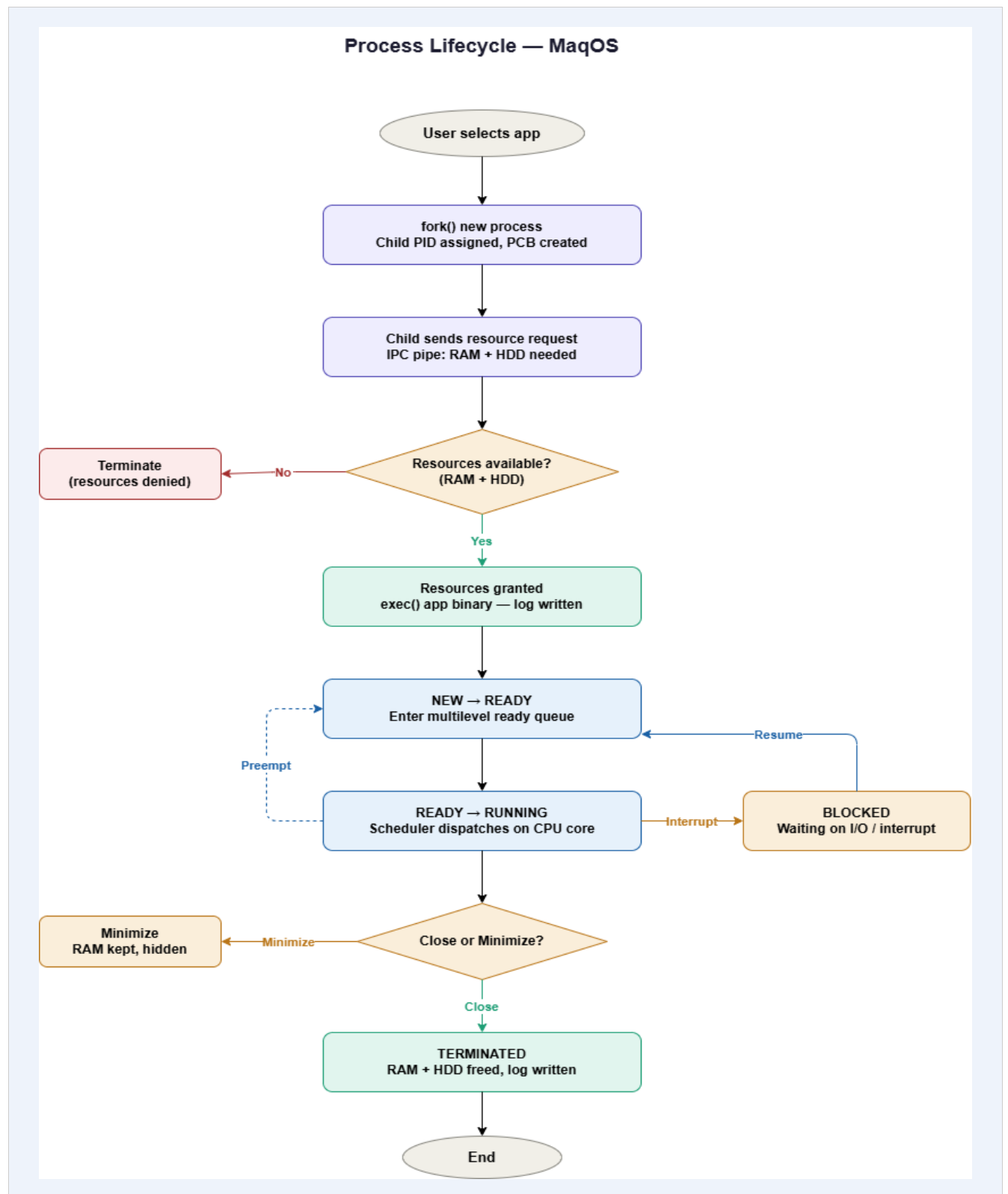
Contents

1. Project Overview.....	8
1.1 Key Objectives.....	8
1.2 Project Identity.....	8
2. System Architecture.....	9
2.1 Layer Summary.....	9
2.2 Source File Breakdown.....	9
2.3 Build System.....	10
3. Boot & Login Sequence.....	11
3.1 Phase 1 — Animated Boot Splash.....	11
3.2 Phase 2 — Lock & Login Screen.....	11
3.3 Hardware Configuration.....	12
4. Desktop Environment & GUI.....	13
4.1 Desktop Overview.....	13
4.2 System Monitor Dashboard.....	13
4.3 Application Icon Grid.....	14
4.4 Taskbar (Glassmorphism).....	14
4.5 Start Menu.....	14
4.6 F1 Instruction Guide.....	15
5. Process Management.....	16
5.1 Process Lifecycle.....	16
5.2 ProcessEntry Structure.....	16
5.3 fork() + exec() with IPC Pipe Handshake.....	16
5.4 Process State Transitions.....	17
5.5 Child Reaping.....	17
6. CPU Scheduling.....	19
6.1 Multilevel Queue Structure.....	19
6.2 Scheduler Operation.....	19
6.3 Priority Aging (Starvation Prevention).....	19
6.4 Full Process Resource Allocation Table.....	19
7. Memory Management.....	21
7.1 RAM Management.....	21
7.2 HDD Management.....	21
7.3 Core Semaphore.....	21
7.4 Real-Time Visualization.....	21
8. Deadlock Detection.....	22

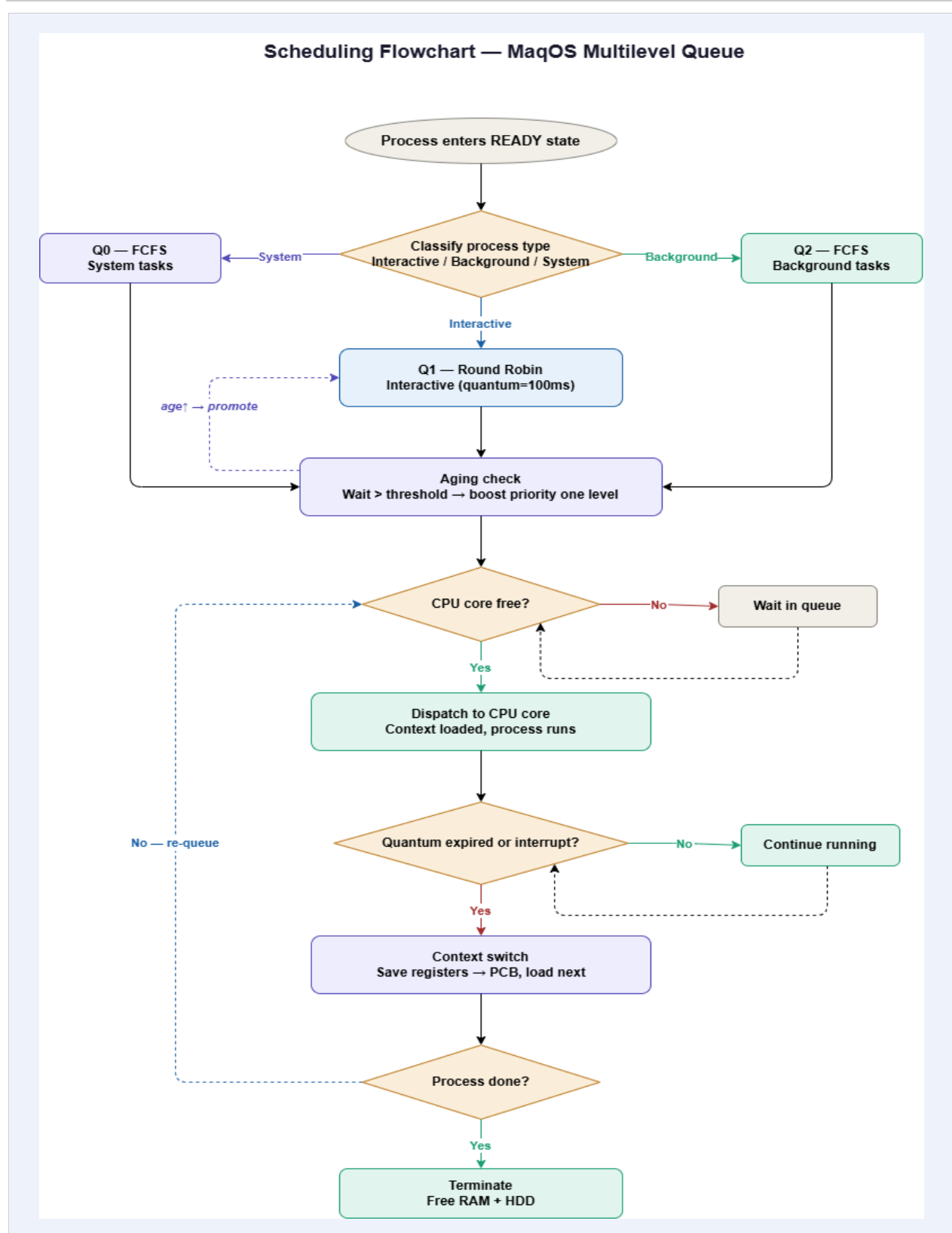
8.1 Simulated Resources.....	22
8.2 Detection Algorithm (DFS on RAG)	22
8.3 I/O Interrupt Simulation.....	22
9. Kernel Mode & User Mode	23
9.1 Mode Switching	23
9.2 Kernel Panel.....	23
9.3 Privilege Comparison.....	24
10. Installed Applications	25
10.1 Productivity.....	25
10.1.1 Calculator.....	25
10.1.2 Notepad (with Auto-Save)	25
10.1.3 Browser.....	26
10.2 System Utilities.....	27
10.2.1 Task Manager	27
10.2.2 Clock.....	28
10.2.3 Calendar	29
10.2.4 Stopwatch	29
10.3 File Management.....	30
10.3.1 File Create	30
10.3.2 File Delete.....	30
10.3.3 File Info	30
10.3.4 File Copy.....	30
10.3.5 File Move (Cut).....	30
10.4 Media	30
10.4.1 Audio Player.....	30
10.4.2 Video Player.....	31
10.5 Games.....	31
10.5.1 Chess.....	32
10.5.2 Snake.....	32
10.5.3 Minesweeper.....	33
10.5.4 Tic Tac Toe	33
10.5.5 Number Guess	33
11. Inter-Process Communication	35
11.1 Pipe-Based Resource Handshake	35
11.2 Signal-Based Process Control.....	35
11.3 System Event Log (maqos_system.log).....	35

12. How to Run MaqOS	36
12.1 Prerequisites (Ubuntu/Debian).....	36
12.2 Build & Launch	36
12.3 Runtime Controls.....	36
13. OS Concepts Implemented	37
14. Additional Screenshots.....	38
14.1 Multiple Apps Running Simultaneously.....	38
14.2 Scheduling & Context Switch Log	38
14.5 Shutdown Screen	38
15. Conclusion	39
15.1 Key Achievements.....	39

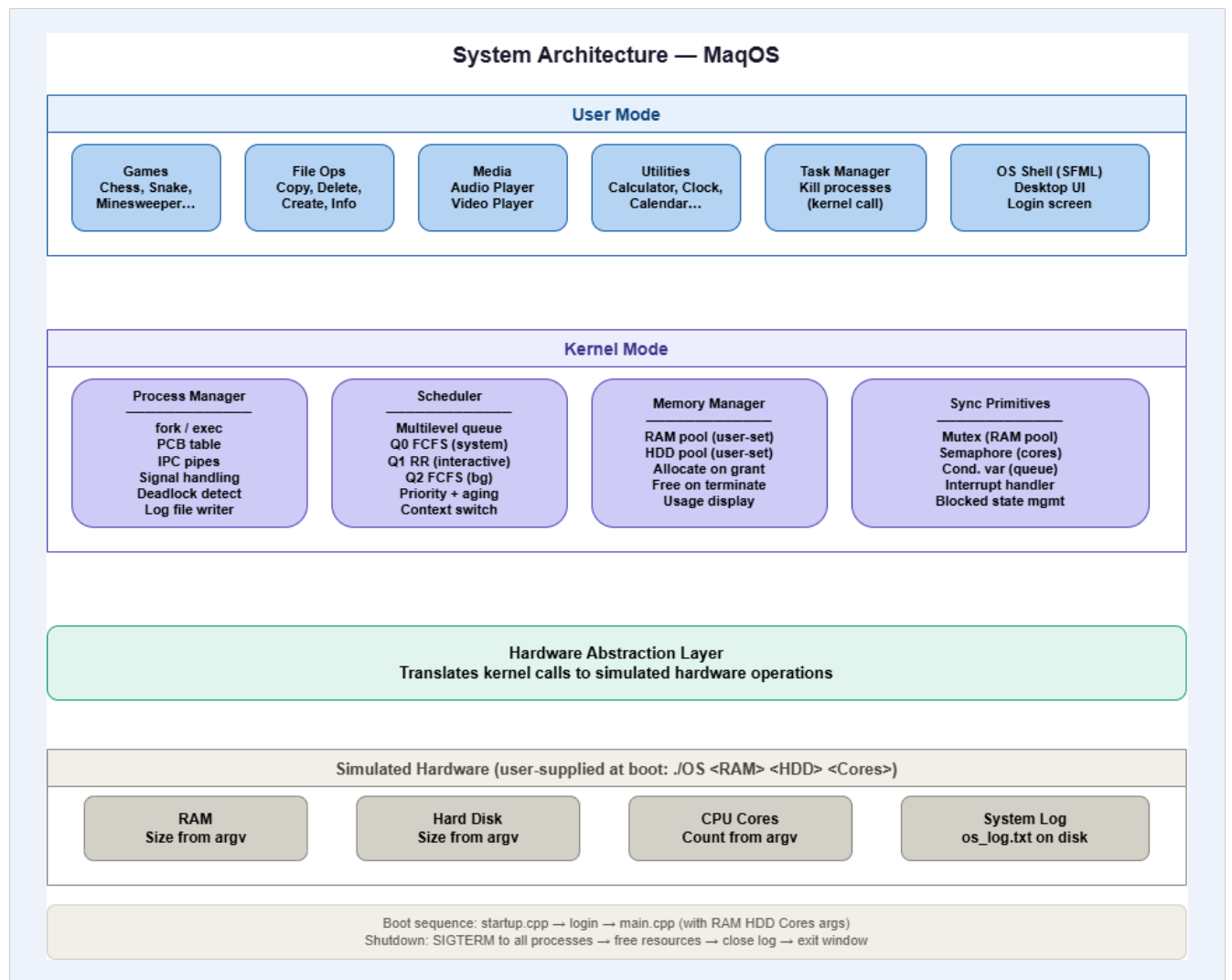
Process LifeCycle



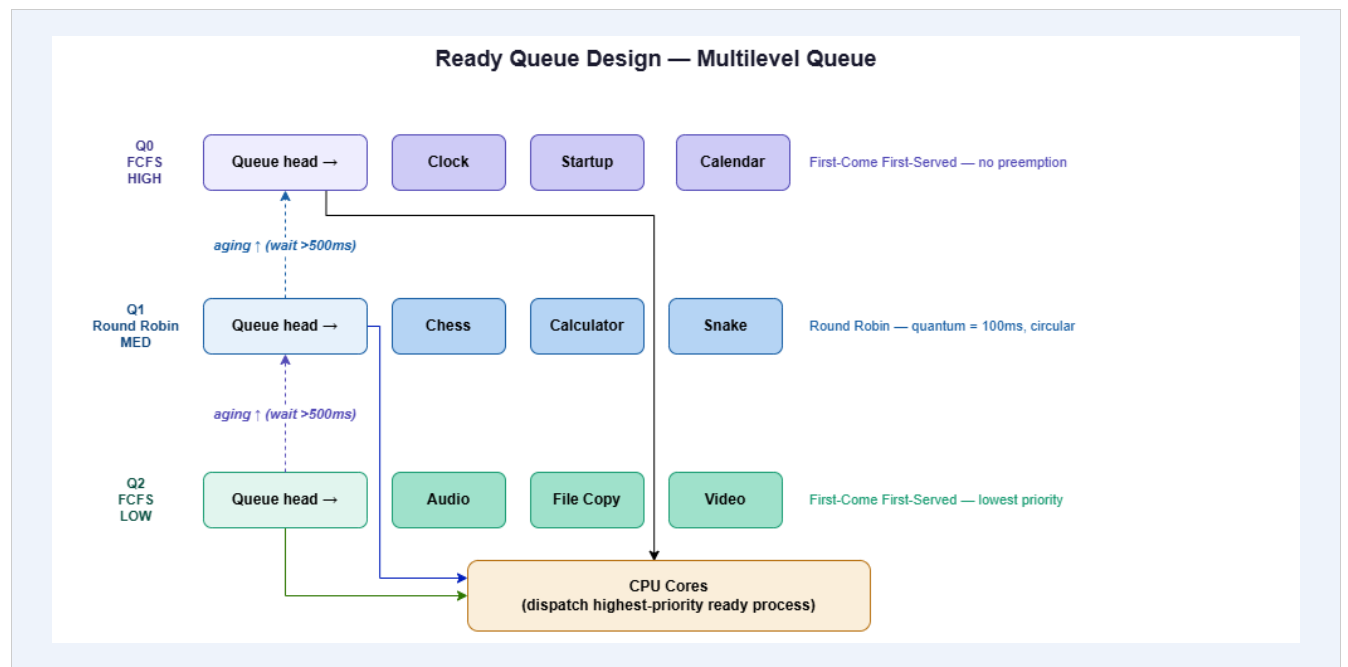
Scheduling FlowChart



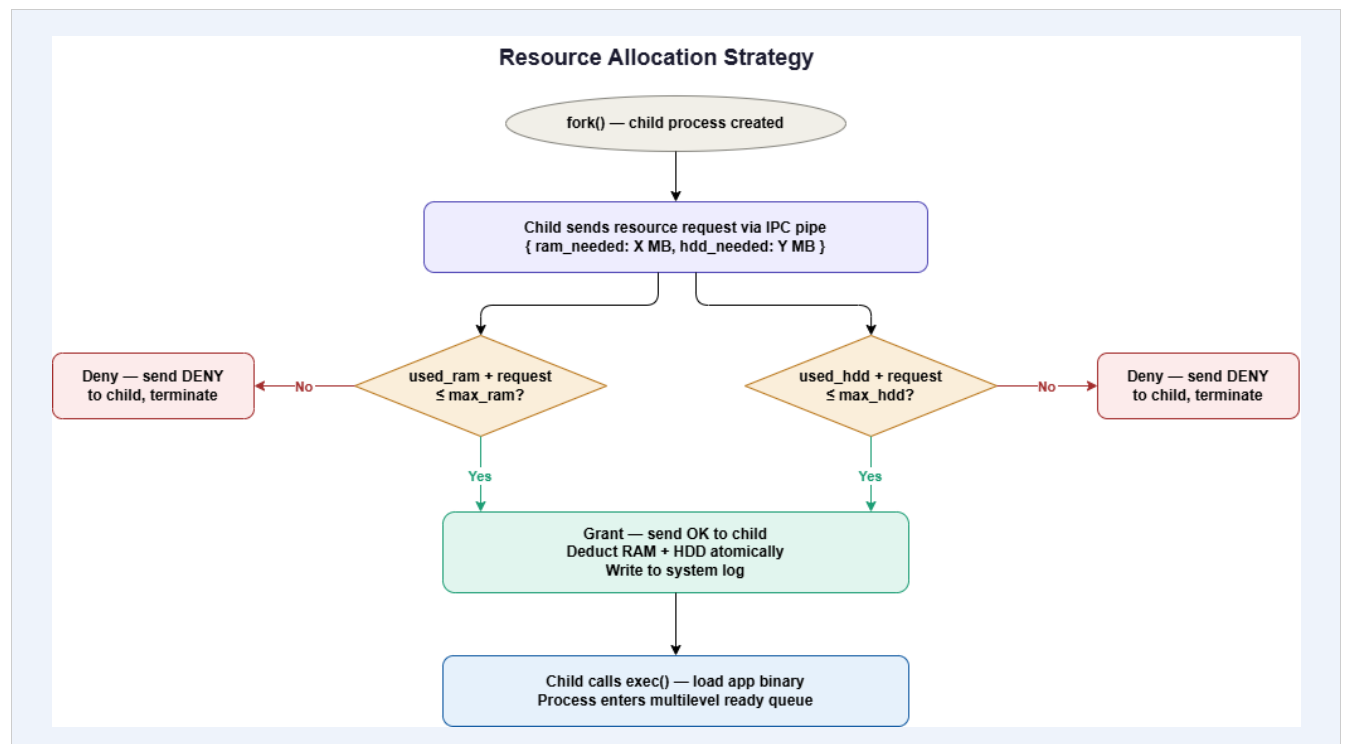
System Architecture



Ready Queue Design



Resource Allocation Strategy



1. Project Overview

MaqOS (Maqsad Operating System) is a fully functional desktop operating system simulation built in C++17 on Ubuntu Linux, using the SFML (Simple and Fast Multimedia Library) as its graphical engine. Developed by M. Taha Rehan, M. Abdullah, and Junaid Haider under the supervision of Mr. Mozeb Ahmed Khan, MaqOS demonstrates core OS concepts — process management, CPU scheduling, memory allocation, file operations, deadlock detection, and inter-process communication — within a polished, graphical desktop environment inspired by Windows 11.

The system launches from the command line with configurable hardware parameters (RAM, HDD, CPU cores) and presents users with a complete desktop experience: an animated boot splash, a secure login screen, a fully interactive desktop with application icons, a centered glassmorphism taskbar, a Start Menu, and a real-time system monitor dashboard.

1.1 Key Objectives

- Simulate real OS behavior using actual Linux system calls (fork, exec, wait, kill, signal).
- Implement a multilevel process scheduler with Round Robin and priority-based queues.
- Provide RAM and HDD resource management using POSIX semaphores and mutex locks.
- Detect potential deadlocks using a Resource Allocation Graph with DFS cycle detection.
- Deliver a graphically rich desktop GUI with 19 interactive applications.
- Demonstrate OS concepts: context switching, process states, aging, kernel/user mode, I/O interrupts.

1.2 Project Identity

Field	Value
OS Name	MaqOS
Full Name	Maqsad Operating System
Version	v1.0 (Nexus)
Course	CL-2006 — Operating Systems Lab
Semester	Spring 2026
Supervisor	Mr. Mozeb Ahmed Khan
Team Members	M. Taha Rehan M. Abdullah Junaid Haider
Default Login	admin / admin
System Log File	maqos_system.log
Technology Stack	C++17, SFML 2.x, POSIX, Ubuntu Linux

2. System Architecture

MaqOS follows a layered multi-process architecture with four distinct layers as shown in Figure 3. The main process (main.cpp) acts as the OS kernel — managing resources, scheduling, and rendering the GUI — while each launched application runs as a fully independent child process spawned via `fork()` and `exec()`.

The architecture mirrors a real OS structure: User Mode applications sit at the top layer, interfacing with Kernel Mode services through system call boundaries. The Kernel Mode layer handles all privileged operations. A Hardware Abstraction Layer translates kernel operations to the simulated hardware parameters supplied at boot.

2.1 Layer Summary

Layer	Components	Role
User Mode	Games, File Ops, Media, Utilities, Task Manager, OS Shell (SFML)	End-user applications launched as child processes
Kernel Mode	Process Manager, Scheduler, Memory Manager, Sync Primitives	Core OS services: fork/exec, MLQ scheduler, RAM pool, semaphores
Hardware Abstraction	HAL — translates kernel calls to simulated hardware	Bridges software OS and simulated hardware configuration
Simulated Hardware	RAM (argv), HDD (argv), CPU Cores (argv), System Log (disk)	User-configured hardware limits set at boot time

2.2 Source File Breakdown

Component	File	Responsibility
Boot & Login	startup.cpp	Animated splash, lock screen, login validation, hardware config prompt
OS Kernel / Desktop	main.cpp	SFML window, GUI, process table, scheduler, RAM/HDD mgmt, IPC
Kernel Panel	K.cpp	Admin view for process control in Kernel Mode
Task Manager	task_manager.cpp	Real-time process list, CPU/RAM stats from /proc filesystem
Notepad	notepad.cpp	Text editor with pthread auto-save every 5 seconds
Calculator	calculator.cpp	Full expression evaluator with themed 4x5 button grid
Browser	browse.cpp	URL launcher + shortcut tiles (Google, YouTube, GitHub...)
Chess	chess.cpp	Two-player chess with move validation and SFML board rendering

Snake	snake.cpp	Classic snake game with smooth animation and high-score file
Minesweeper	minesweeper.cpp	Full Minesweeper with mine gen, recursive reveal, flag system
Tic Tac Toe	ticTacToc.cpp	Two-player Tic Tac Toe with win-line detection
Number Guess	numberGuess.cpp	Guessing game with higher/lower hints and attempt counter
File Utilities	fileCreate/Delete/Info/copy/cut .cpp	OS-level file CRUD operations via POSIX APIs
Media	audioplayer.cpp, VideoPlayer.cpp	Audio/video playback via system media tools
Clock / Calendar	clock.cpp, simple_calendar.cpp	Auto-started system processes on boot
Stopwatch	stopWatch.cpp	Precision timer with SFML display
Shutdown	shutdown.cpp	Graceful shutdown with goodbye screen

2.3 Build System

A single Makefile compiles every .cpp file in src/ into a corresponding binary in bin/. The check-deps target auto-installs all required system packages on first build.

```
make all                # Build all + auto-install deps
./bin/startup           # Full boot (splash -> login -> desktop)
./bin/main 4 256 8      # Direct launch: 4GB RAM, 256GB HDD, 8
Cores
make clean              # Remove all binaries
```

Required packages: libsfml-dev, ffmpeg, vlc, zenity, pkg-config, libgtkmm-3.0-dev, build-essential.

3. Boot & Login Sequence

The startup sequence simulates a real OS boot: animated splash -> secure login -> hardware configuration -> desktop launch.

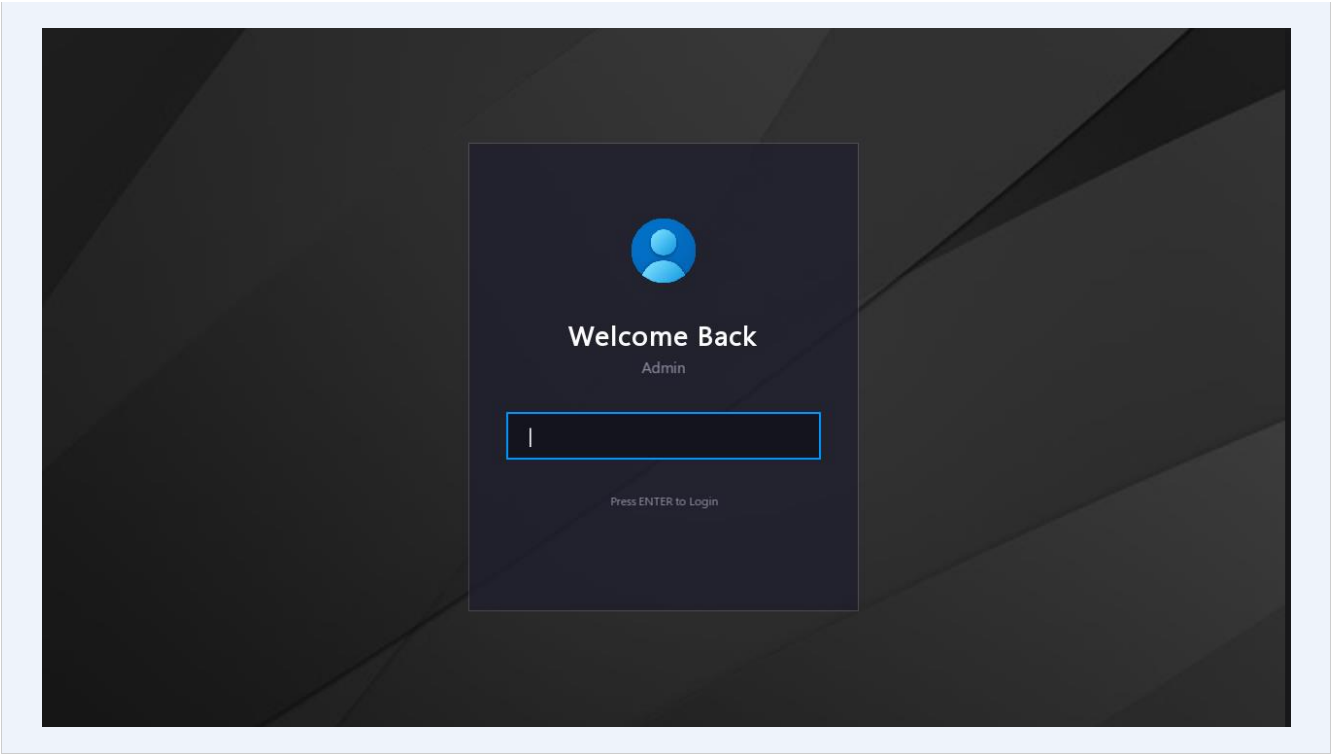
3.1 Phase 1 — Animated Boot Splash

The MaqOS branding image fades in and out over 4.5 seconds. A progress bar fills from 0-100% synchronized with elapsed time, and a "Loading MaqOS Kernel..." label provides feedback. After the splash, the hardware parameters are read from command-line arguments.



3.2 Phase 2 — Lock & Login Screen

A stylized lock screen (lock.jpg background) presents a centered login panel. Username and password fields with character masking are rendered in SFML. Default credentials: admin / admin. Failed attempts display an error message. Successful login transitions to the desktop.



3.3 Hardware Configuration

Hardware limits are passed as command-line arguments and stored as global constants used throughout the session for all resource allocation decisions:

```
./bin/main <RAM_GB> <HDD_GB> <Cores>
```

Parameter	Effect in MaqOS
RAM_GB	Sets MAX_RAM (in MB). Binary semaphore protects allocation. Apps denied if used_ram + request > MAX_RAM.
HDD_GB	Sets MAX_HDD (in MB). Atomic counter tracks usage. Apps denied if used_hdd + request > MAX_HDD.
Cores	Initialises counting semaphore core_sem. Limits concurrent processes to this number.

4. Desktop Environment & GUI

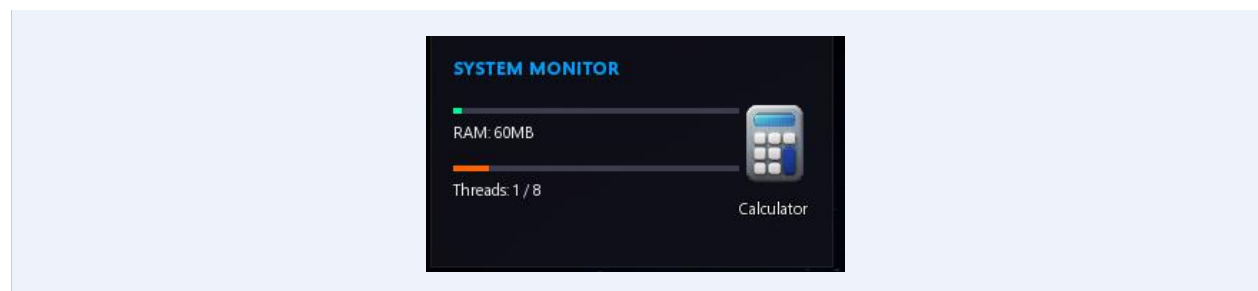
The MaqOS desktop is rendered entirely in SFML at 1280x720 (resizable). It presents a modern OS interface with wallpaper, application icons, a centered glassmorphism taskbar, and an integrated live system monitor.

4.1 Desktop Overview



4.2 System Monitor Dashboard

A frosted-glass panel (top-left) shows real-time RAM and CPU usage bars updated every frame. RAM fills proportionally in green; CPU/thread usage fills in orange. The label reads "SYSTEM MONITOR" in blue.

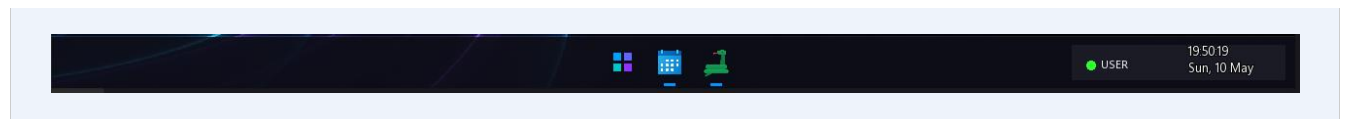


4.3 Application Icon Grid

19 application icons (64x64px) are arranged in a 6-per-row grid. Hovering highlights with a semi-transparent overlay. Left-click launches the app via `fork()` + `exec()`. Right-click minimizes/restores a running instance.

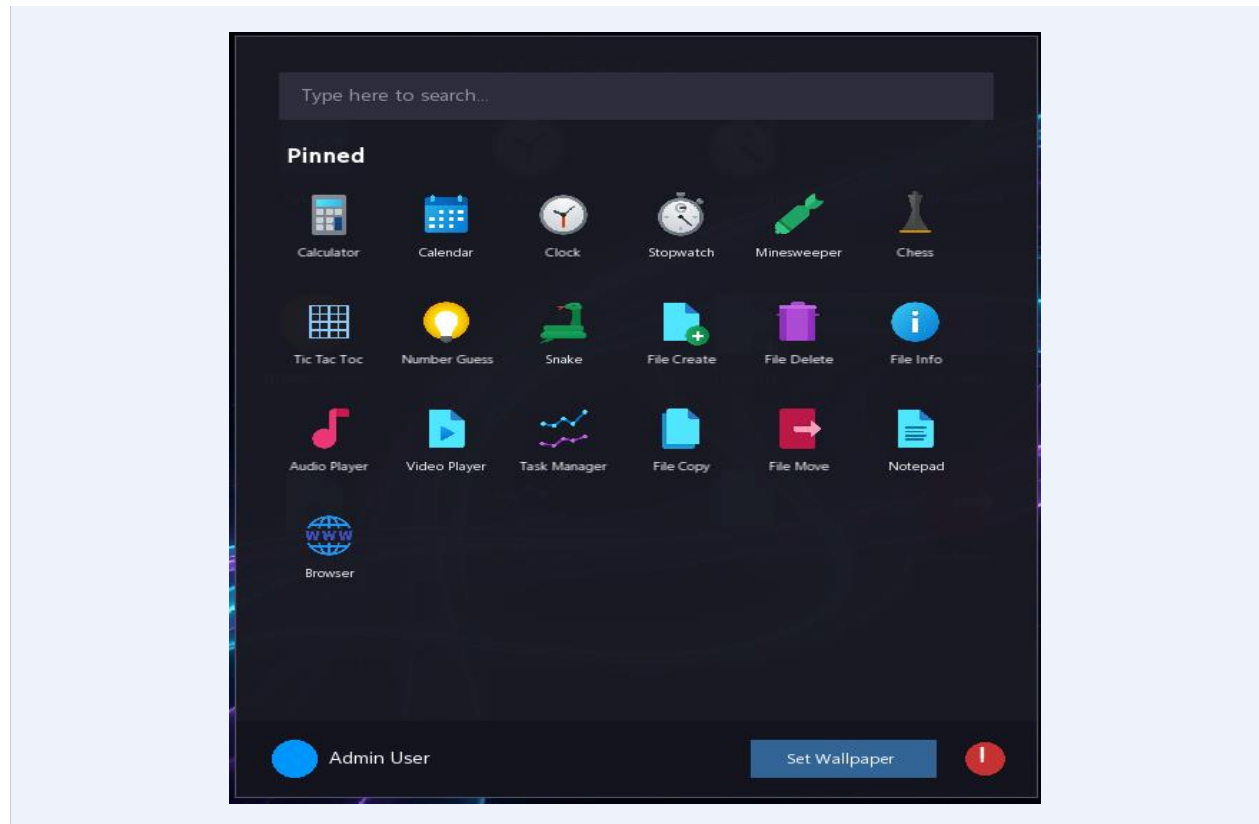
4.4 Taskbar (Glassmorphism)

A 52px taskbar at the screen bottom has three zones: a centered app dock (Start Button + running app icons with blue indicators), a system tray (clock + date + mode dot), and a Kernel Mode toggle button. Running apps show a blue underline indicator. Right-clicking a taskbar app icon sends `SIGTERM` to close it.



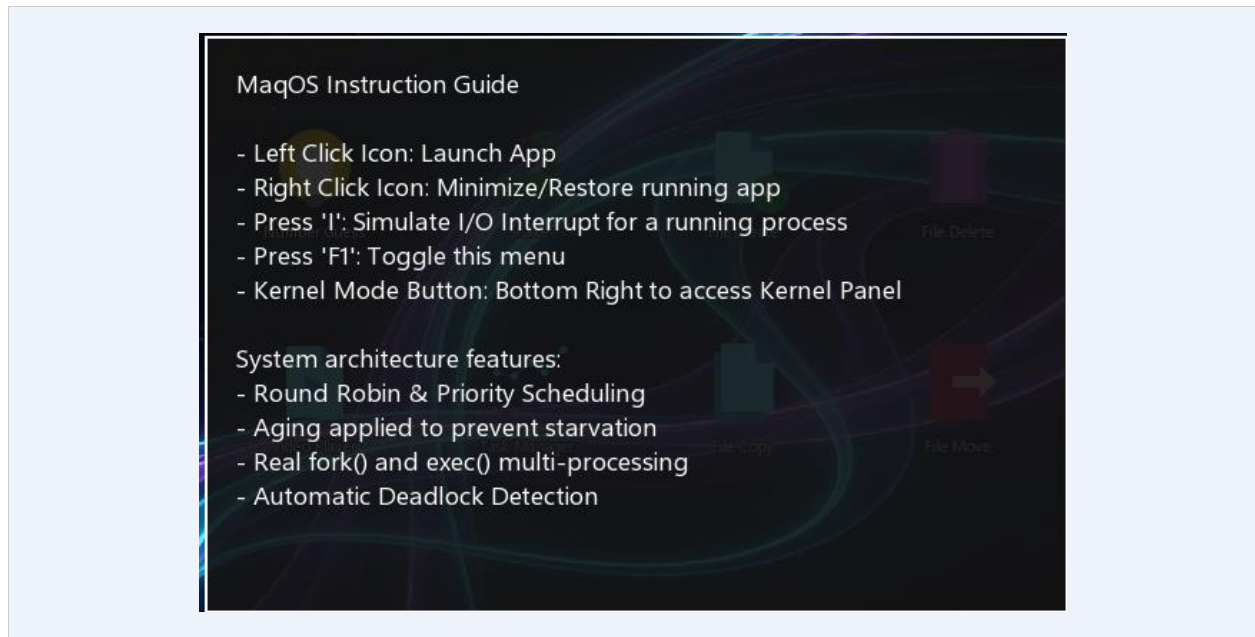
4.5 Start Menu

Clicking Start opens a 560x640px Windows 11-style panel with a search bar, 6-column app grid, user bar ("Admin User"), Set Wallpaper button (opens native Zenity file dialog), and a shutdown power button. Clicking outside closes the menu.



4.6 F1 Instruction Guide

Pressing F1 toggles a full help overlay listing all keyboard shortcuts and mouse interactions. Key I simulates an I/O interrupt on the current running process.



5. Process Management

5.1 Process Lifecycle

The Process Lifecycle diagram (Figure 1) captures the complete journey of every MaqOS process from launch to termination. Key stages:

- User selects app -> `fork()` creates child; child PID assigned, PCB entry created.
- IPC pipe handshake: child sends RAM + HDD request; parent grants or denies.
- If granted: `exec()` loads app binary; process enters multilevel ready queue.
- Scheduler dispatches process to CPU core (READY -> RUNNING).
- Running process may be interrupted (-> BLOCKED, auto-resumes after 3s) or minimized (SIGSTOP).
- On close: SIGTERM sent; RAM + HDD freed; log written; entry removed from process table.

5.2 ProcessEntry Structure

Field	Type	Description
pid	pid_t	Linux Process ID assigned by the kernel via <code>fork()</code>
name	string	Human-readable application name (e.g., "Calculator")
ram_mb	int	RAM allocated to this process in megabytes
hdd_mb	int	HDD space allocated to this process in megabytes
priority	int	Scheduling priority — 0 = highest, 5 = lowest
wait_ticks	int	Ticks spent waiting; incremented by aging algorithm
arrival_time	time_t	Unix timestamp of process creation
state	enum	READY RUNNING MINIMIZED BLOCKED TERMINATED

5.3 `fork()` + `exec()` with IPC Pipe Handshake

The `fork_and_exec()` function creates every application process using two anonymous pipes for resource negotiation before `exec()`:

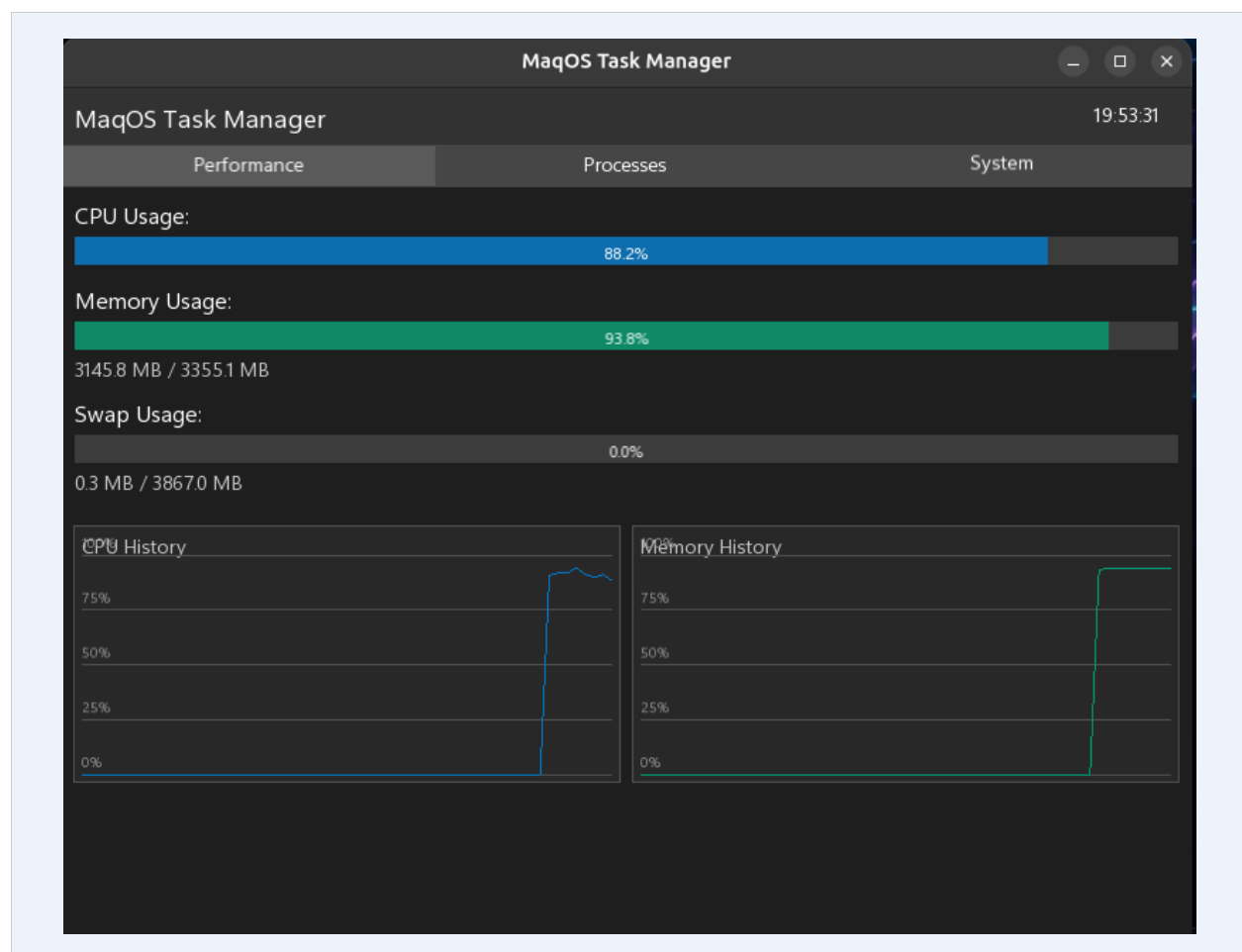
1. `fork()` creates the child. Child calls `setsid()` and closes all inherited file descriptors except the IPC pipes.
2. Child writes `[ram_mb, hdd_mb]` to `req_pipe`. Parent reads this request.
3. Parent checks `allocate_ram()` and `allocate_hdd()`. If both succeed: ACK = 1 (grant). Otherwise: ACK = 0 (deny), log "RESOURCE DENIED".
4. Parent writes ACK to `ack_pipe`. Child reads ACK.
5. If ACK = 1: child calls `execlp()` to replace itself with the app binary. If ACK = 0: child calls `_exit(0)` silently.
6. Parent adds `ProcessEntry` to `process_table` and queues it in the appropriate scheduling queue.

5.4 Process State Transitions

State	Trigger	Mechanism
READY	Resources granted, process queued	Added to MLQ; waits for scheduler dispatch
RUNNING	Scheduler dispatches process	State updated; SIGCONT sent if was stopped
MINIMIZED	User right-clicks icon / taskbar	SIGSTOP freezes process; RAM kept allocated
BLOCKED	Key 'I' — simulated I/O interrupt	SIGSTOP; detached thread resumes via SIGCONT after 3s
TERMINATED	App exits or SIGTERM received	waitpid() detects; RAM + HDD freed; sem_post(core_sem)

5.5 Child Reaping

reap_children() runs every frame using waitpid(WNOHANG) to non-blockingly detect exited children. Terminated entries are freed from the process table, preventing zombie processes.



MaqOS Task Manager				
MaqOS Task Manager				
19:53:54				
Performance		Processes		System
Name	PID	CPU %	Memory	Status
/usr/lib/systemd/systemd	1	257.5	15.8 MB	Running
kthreadd	2	1.5	0 KB	Running
pool_workqueue_release	3	0.0	0 KB	Running
kworker/R-rcu_gp	4	0.0	0 KB	Running
kworker/R-sync_wq	5	0.0	0 KB	Running
kworker/R-kvfree_rcu_reclaim	6	0.0	0 KB	Running
kworker/R-slub_flushwq	7	0.0	0 KB	Running
kworker/R-netns	8	0.0	0 KB	Running
kworker/0:1-cgroup_free	11	3.5	0 KB	Running
kworker/u512.0-ipv6_addrconf	12	0.0	0 KB	Running
kworker/R-mm_percpu_wq	13	0.0	0 KB	Running
ksoftirqd/0	14	9.0	0 KB	Running
rcu_preempt	15	64.0	0 KB	Running
rcu_exp_par_gp_kthread_worker/1	16	0.5	0 KB	Running
rcu_exp_gp_kthread_worker	17	1.0	0 KB	Running
migration/0	18	0.5	0 KB	Running
idle_inject/0	19	0.0	0 KB	Running
cpuhp/0	20	0.0	0 KB	Running
cpuhp/1	21	0.0	0 KB	Running
Kill Process	22	0.0	0 KB	Running

6. CPU Scheduling

MaqOS implements a three-level Multilevel Queue (MLQ) scheduler running on a dedicated pthread. The Scheduling Flowchart (Figure 2) and Ready Queue Design (Figure 4) illustrate the complete decision logic.

6.1 Multilevel Queue Structure

Level	Queue Type	Policy	Applications
Q0 — System	FCFS Queue	First-Come First-Served, no preemption	Clock, Calendar, TaskManager, Startup
Q1 — Interactive	Priority Queue (min-heap)	Priority + Round Robin (2s quantum)	Calculator, Games, Notepad, Browser, VideoPlayer
Q2 — Background	FCFS Queue	First-Come First-Served, lowest priority	AudioPlayer, FileCopy, FileMove, File utilities

6.2 Scheduler Operation

The `maqos_scheduler()` pthread runs continuously, waking every 500ms or when notified via condition variable. Each cycle:

7. `apply_aging()` increments `wait_ticks` for all queued processes and boosts priority every `AGING_THRESHOLD` (10) ticks.
8. Queue priority: Q0 checked first -> Q1 -> Q2. Highest-priority ready process is selected.
9. Context switch logged: "CONTEXT SWITCH: AppA -> AppB".
10. For Q1 processes: a detached thread simulates Round Robin quantum expiry after 2 seconds, logging "RR QUANTUM EXPIRED".
11. Process state updated to RUNNING in the process table.

6.3 Priority Aging (Starvation Prevention)

To prevent indefinite starvation of lower-priority processes, every waiting process has its `wait_ticks` incremented each scheduler tick. When `wait_ticks` is a multiple of `AGING_THRESHOLD` (10), the priority is decremented by 1 (moving toward 0 = highest). Each promotion is logged: "AGING: AppName priority boosted to N".

6.4 Full Process Resource Allocation Table

#	Application	Queue Level	RAM (MB)	HDD (MB)	Priority
1	Calculator	Q1 — Interactive	80	5	2
2	Calendar	Q0 — System (Auto)	60	5	0

3	Clock	Q0 — System (Auto)	40	2	0
4	Stopwatch	Q1 — Interactive	50	2	2
5	Minesweeper	Q1 — Interactive	120	10	2
6	Chess	Q1 — Interactive	150	15	2
7	Tic Tac Toe	Q1 — Interactive	80	8	2
8	Number Guess	Q1 — Interactive	60	5	2
9	Snake	Q1 — Interactive	100	8	2
10	File Create	Q2 — Background	70	10	4
11	File Delete	Q2 — Background	70	5	4
12	File Info	Q2 — Background	60	5	4
13	Audio Player	Q2 — Background	100	30	4
14	Video Player	Q1 — Interactive	300	100	2
15	Task Manager	Q0 — System	120	5	1
16	File Copy	Q2 — Background	150	50	4
17	File Move	Q2 — Background	150	50	4
18	Notepad	Q1 — Interactive	100	20	2
19	Browser	Q2 — Background	200	50	4

7. Memory Management

The Resource Allocation Strategy (Figure 5) shows the complete flow of RAM and HDD allocation in MaqOS. All resource management is atomic and thread-safe via POSIX semaphores.

7.1 RAM Management

RAM is configured at boot (e.g., 4 GB = 4096 MB). A global `used_ram` counter is protected by a binary semaphore (`ram_sem`):

- `allocate_ram(mb)`: `sem_wait` -> check `used_ram + mb <= MAX_RAM` -> increment -> `sem_post`. Returns false if insufficient.
- `free_ram(mb)`: `sem_wait` -> decrement `used_ram` (floored at 0) -> `sem_post`.

The IPC pipe handshake ensures resources are negotiated before the child process begins executing. If RAM or HDD is insufficient, the child exits silently and the denial is logged.

7.2 HDD Management

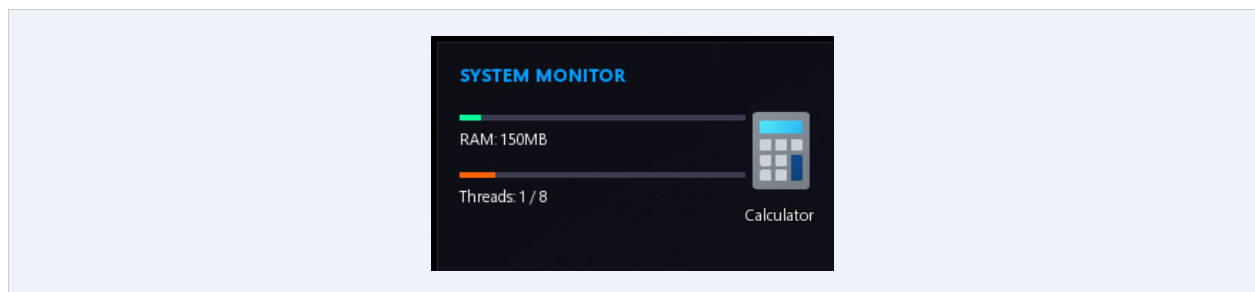
HDD space (up to 256 GB = 262,144 MB) uses an `atomic<int>` (`used_hdd`) protected by `hdd_mutex`. Both RAM and HDD must clear their checks for any process to launch. On termination, both are freed atomically in `reap_children()`.

7.3 Core Semaphore

A counting semaphore (`core_sem`) initialized with the configured core count limits concurrent processes. It is acquired on dispatch and released on termination, ensuring MaqOS never runs more processes than the configured core count.

7.4 Real-Time Visualization

The desktop System Monitor dashboard shows live RAM (green bar) and CPU/thread (orange bar) usage updated every frame, providing an always-visible view of system resource consumption.



8. Deadlock Detection

MaqOS implements a Resource Allocation Graph (RAG)-based deadlock detection system running as a background pthread, scanning every 5 seconds.

8.1 Simulated Resources

ID	Resource	Available Instances
0	FileSystem	2
1	AudioDevice	1
2	Display	3

8.2 Detection Algorithm (DFS on RAG)

12. Build RAG: edge from process $i \rightarrow$ process j if i requests a resource that j holds.
13. DFS cycle detection: if any back-edge found (node on DFS stack revisited), a cycle — and thus deadlock — exists.
14. Alert: deadlocked process names printed to stdout and logged: "DEADLOCK DETECTED among: AppA AppB".

8.3 I/O Interrupt Simulation

Pressing key 'I' triggers `simulate_io_interrupt()` on the first RUNNING process. The process moves to BLOCKED state (SIGSTOP sent). A detached thread auto-resolves after 3 seconds (SIGCONT sent + log entry "INTERRUPT RESOLVED"). This simulates a process blocking on a device I/O request and later resuming.

9. Kernel Mode & User Mode

MaqOS implements a two-mode privilege system analogous to ring-0 / ring-3 in real CPUs.

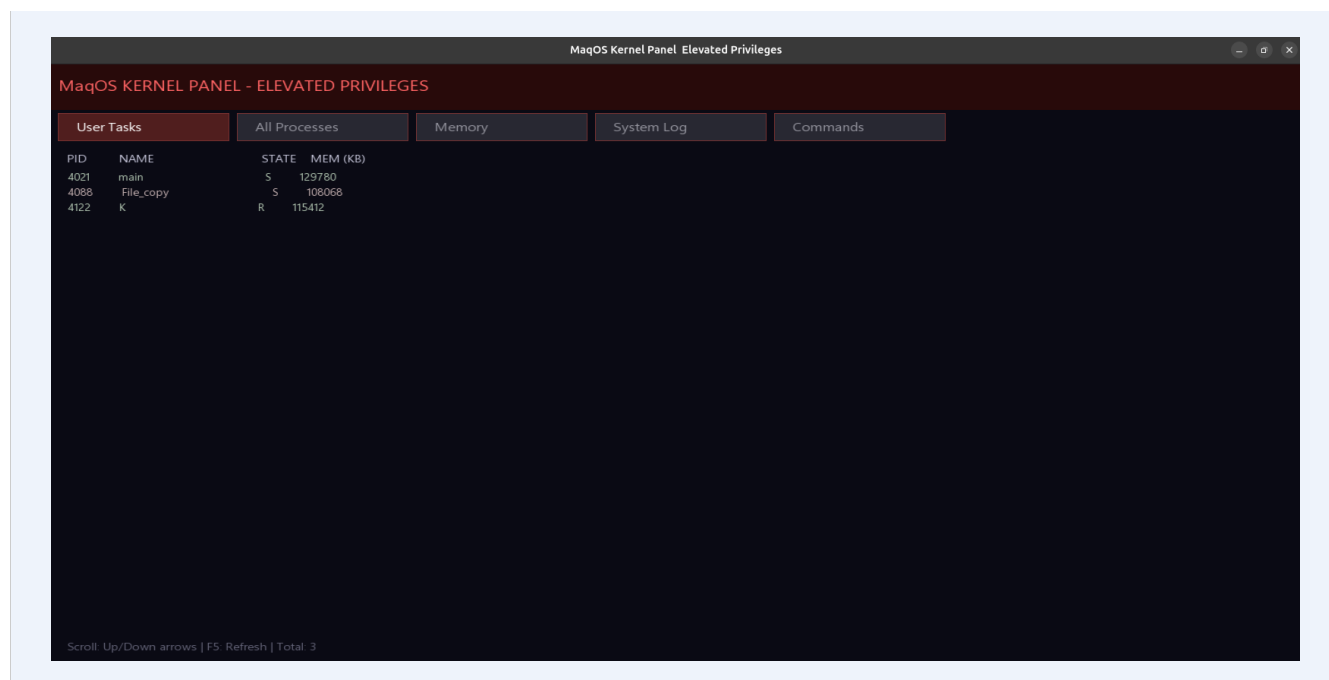
9.1 Mode Switching

The system tray shows a green dot (User Mode) or red dot (Kernel Mode). Clicking the Kernel Mode button in the taskbar toggles modes with a 500ms debounce. Each switch is logged to `maqos_system.log`.



9.2 Kernel Panel

Entering Kernel Mode auto-launches the Kernel Panel (`./bin/K`) as a privileged child process. It provides admin-level access not available in User Mode.



9.3 Privilege Comparison

Action	User Mode	Kernel Mode
Launch applications	Yes	Yes
Minimize / restore own apps	Yes	Yes
Close apps via taskbar	Yes	Yes
View process list	Task Manager	Kernel Panel
Force-kill any process	No	Yes (SIGKILL)
Adjust process priorities	No	Yes
View resource allocation matrix	No	Yes
Kernel Panel access	No	Auto-launched

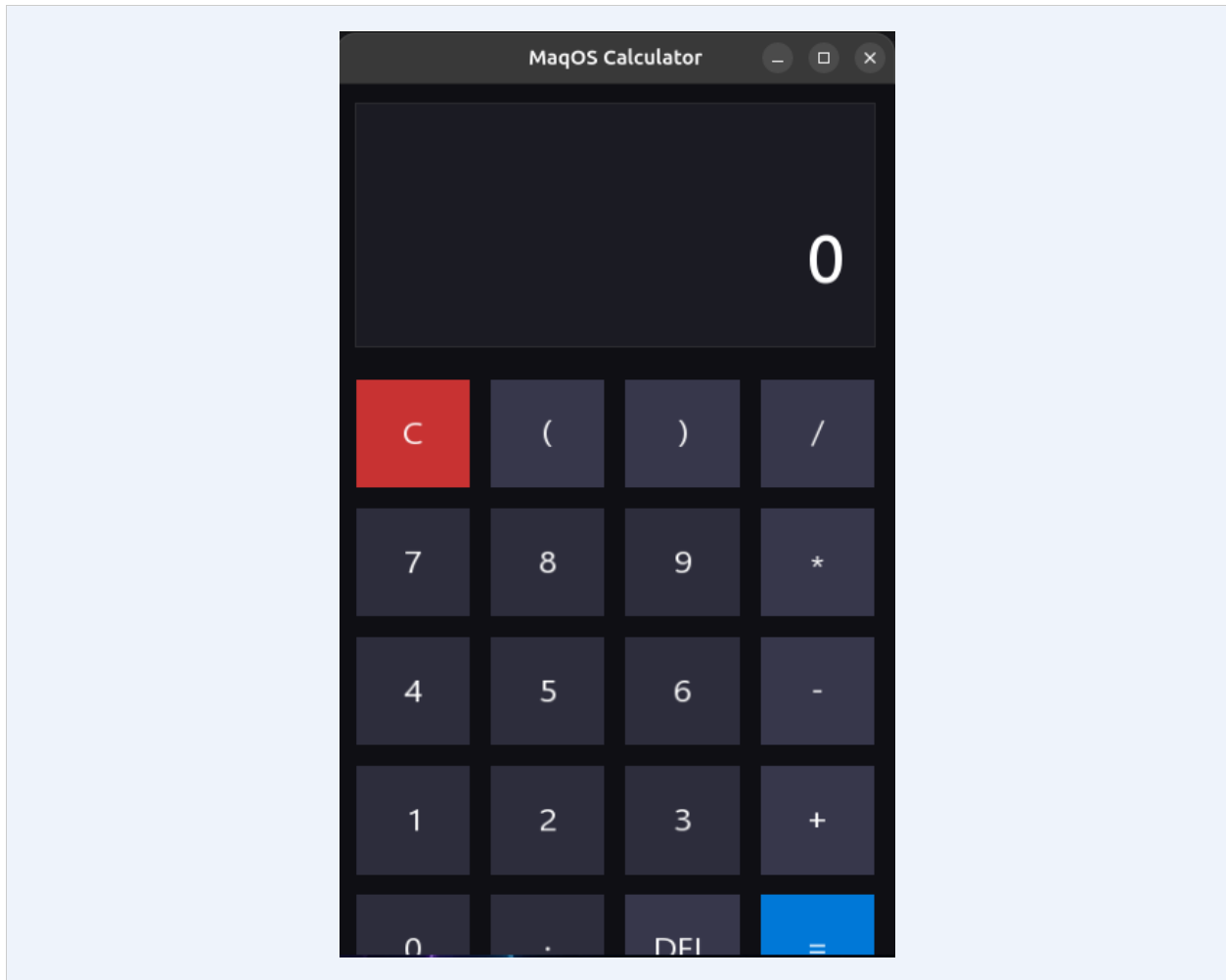
10. Installed Applications

MaqOS ships 19 built-in applications across five categories. Every application is a standalone SFML binary launched as an independent child process.

10.1 Productivity

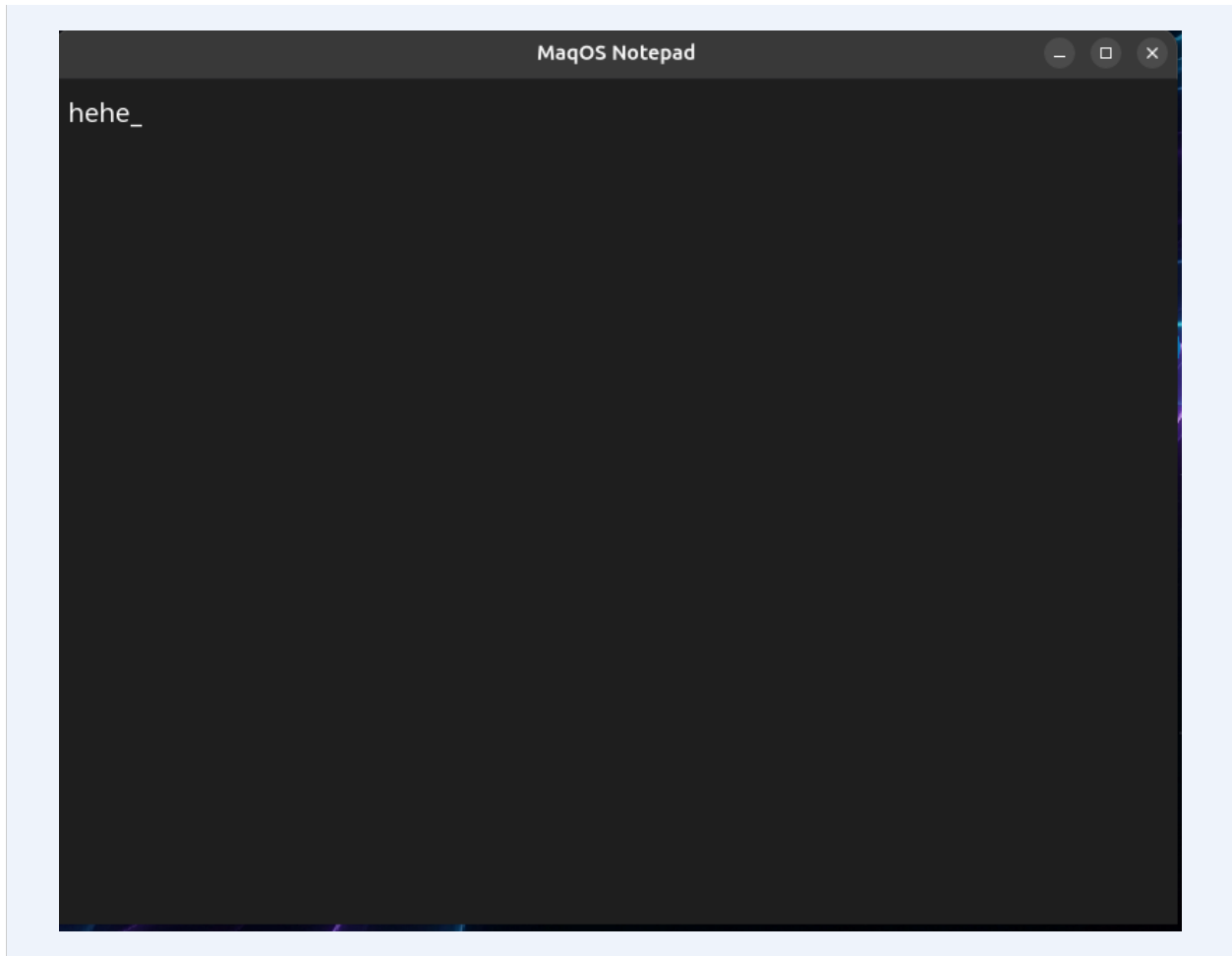
10.1.1 Calculator

Full expression calculator — 4x5 button grid with +, -, x, /, parentheses, DEL, clear. Large themed display shows expression and result.



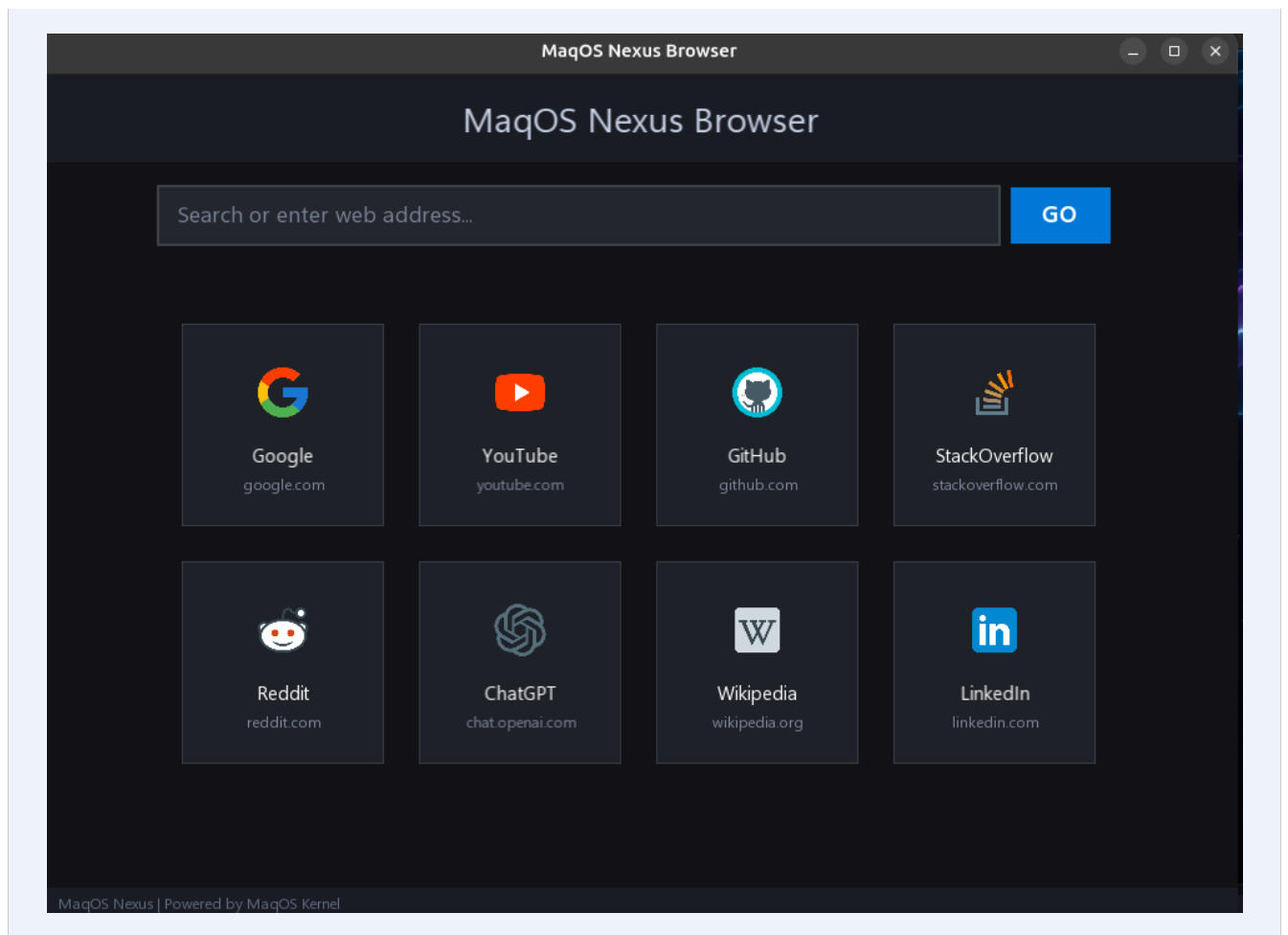
10.1.2 Notepad (with Auto-Save)

Plain-text editor with real-time character input. A pthread auto-saves the buffer to `maqos_notepad_save.txt` every 5 seconds using `pthread_mutex`. Final save on close.



10.1.3 Browser

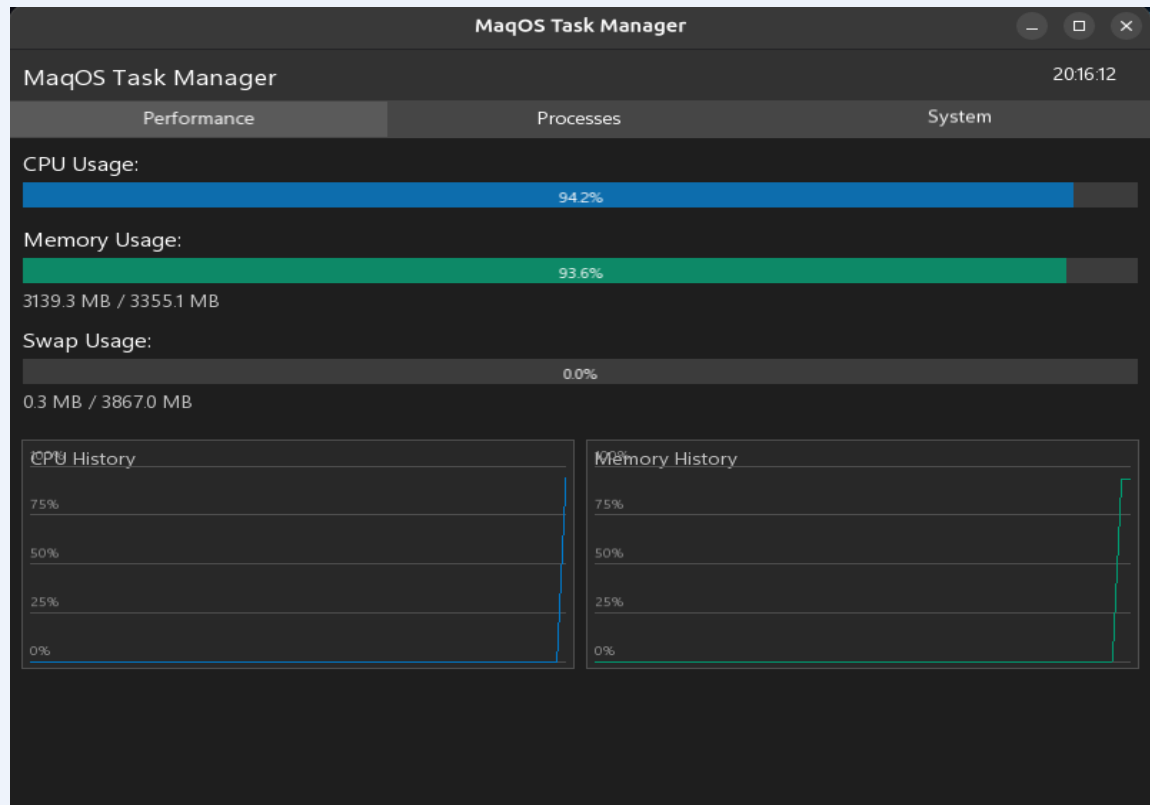
GUI web launcher — URL input bar + shortcut tile grid for Google, GitHub, YouTube, Wikipedia, LinkedIn, Reddit, Stack Overflow, ChatGPT. Opens URLs via xdg-open (Linux).



10.2 System Utilities

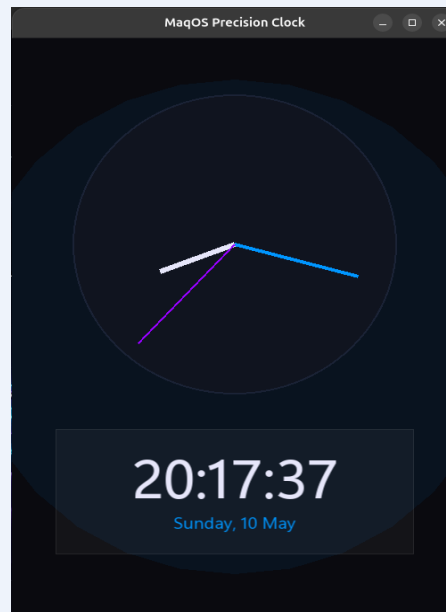
10.2.1 Task Manager

Reads real process data from Linux /proc. Live-updating table: PID, name, CPU%, memory (KB), status. System-wide RAM and swap totals at top. pthread refresh loop.



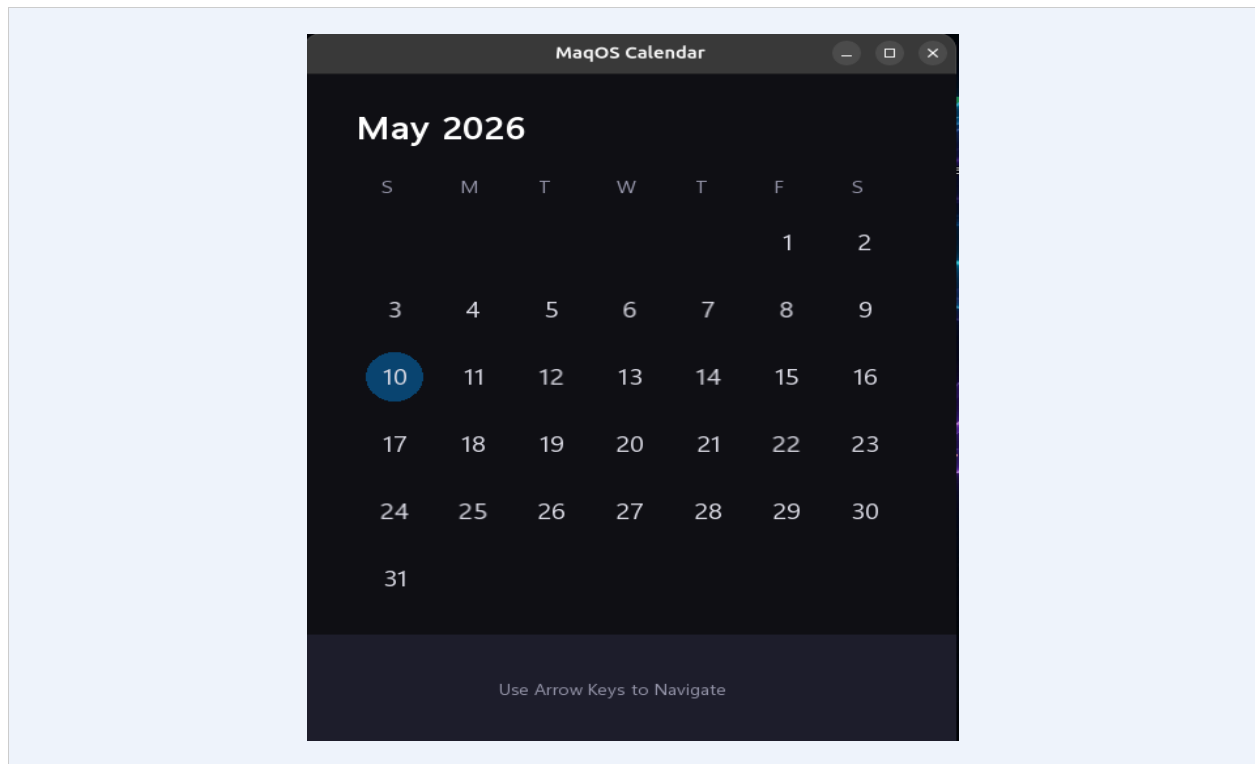
10.2.2 Clock

Auto-started on boot. Real-time clock display updated every second.



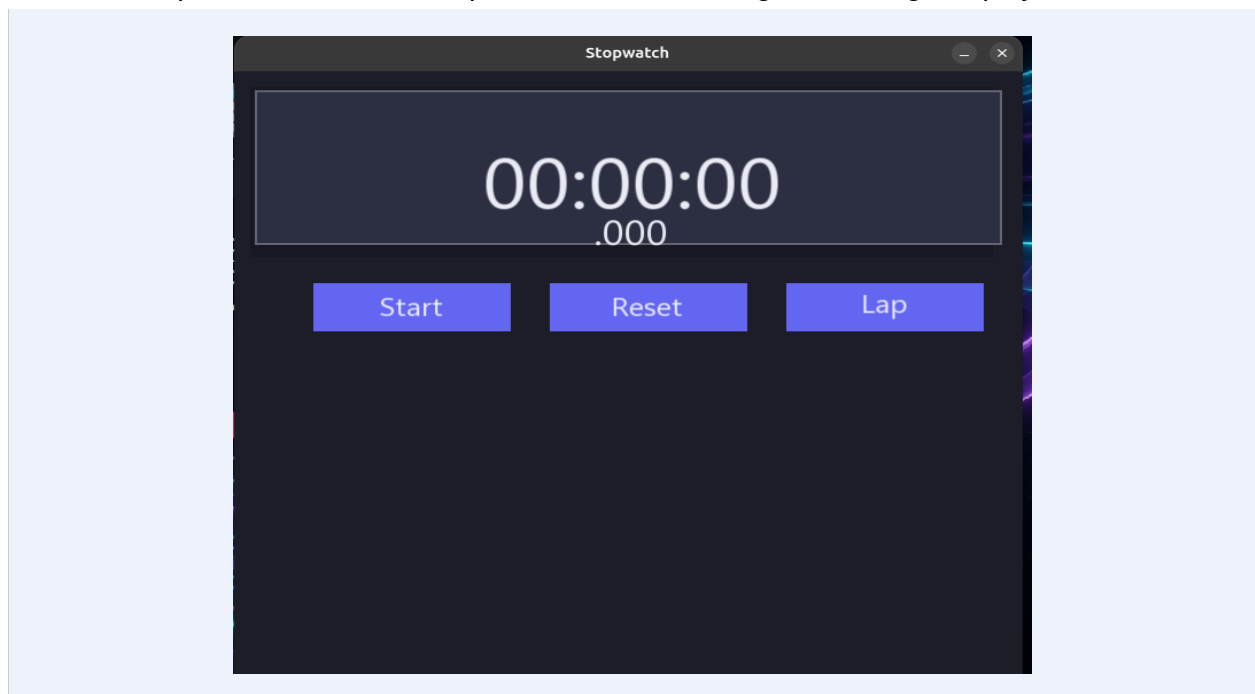
10.2.3 Calendar

Auto-started on boot. Monthly calendar grid with current date highlighted.



10.2.4 Stopwatch

Precision stopwatch with Start / Stop / Reset controls. Large SFML digit display.



10.3 File Management

10.3.1 File Create

GUI interface to create new files using C++ fstream. User enters filename and content.

10.3.2 File Delete

Lists current directory files. User selects target; confirmed deletion via remove()/unlink().

10.3.3 File Info

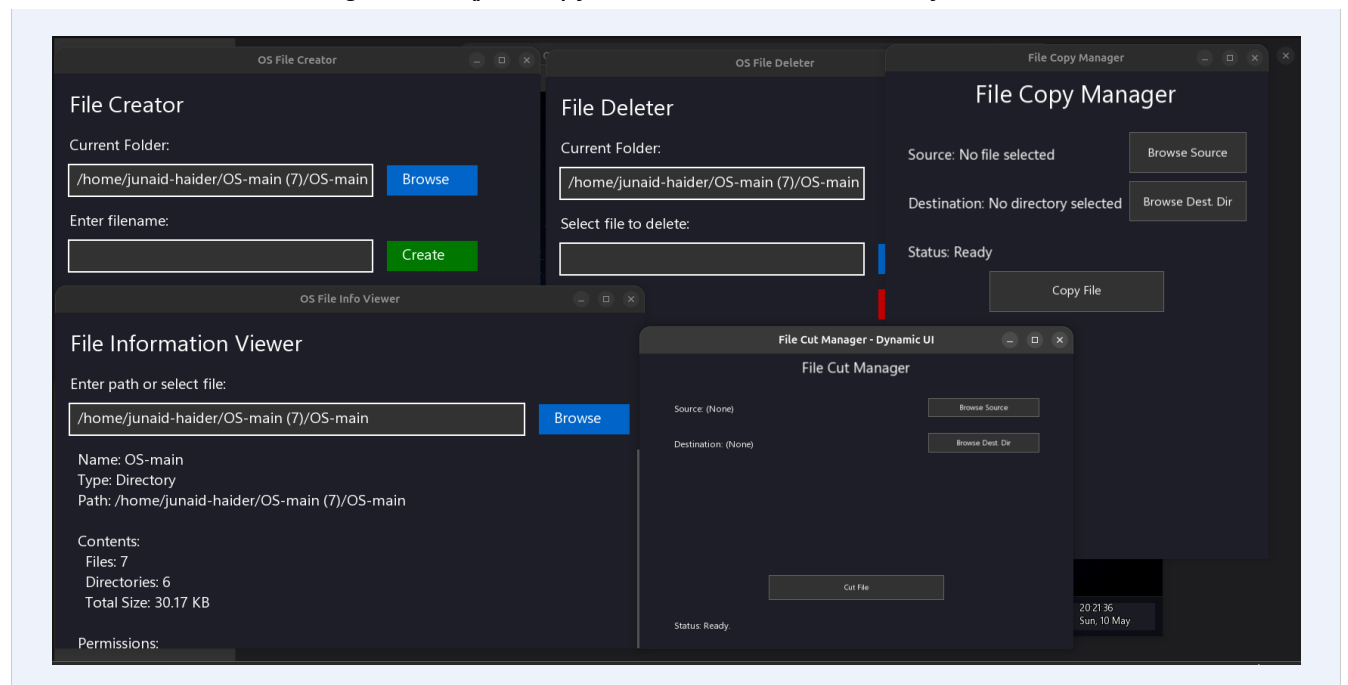
Shows file metadata: size, creation/modified dates, permissions — retrieved via stat() system call.

10.3.4 File Copy

Copies source file to destination using buffered file I/O for efficiency.

10.3.5 File Move (Cut)

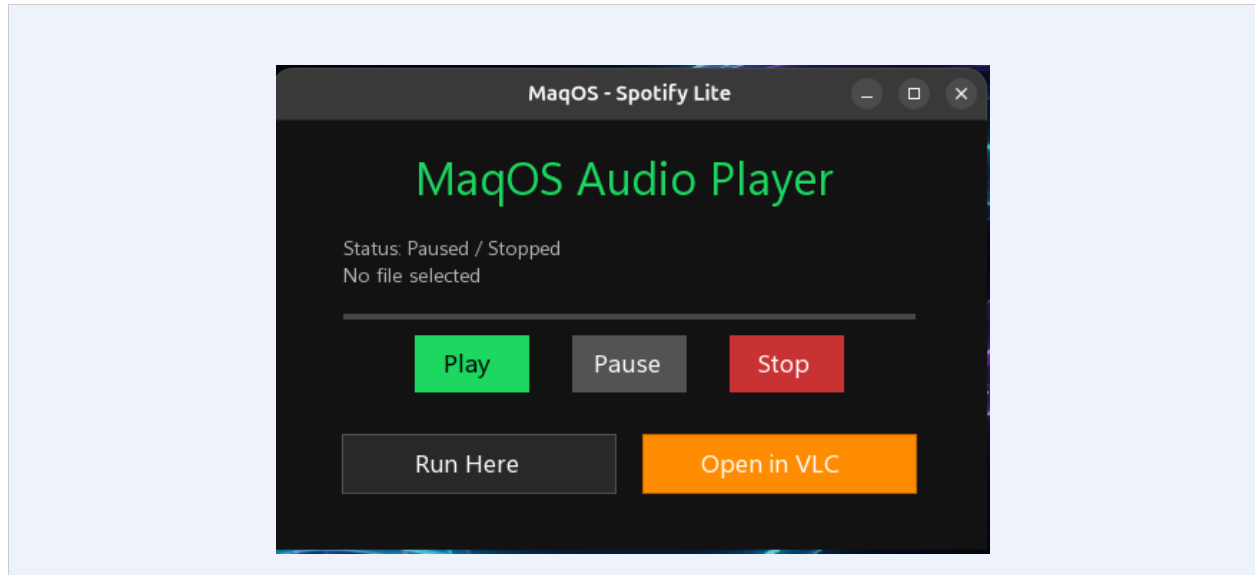
Moves/renames files using rename() or copy-then-delete for cross-filesystem moves.



10.4 Media

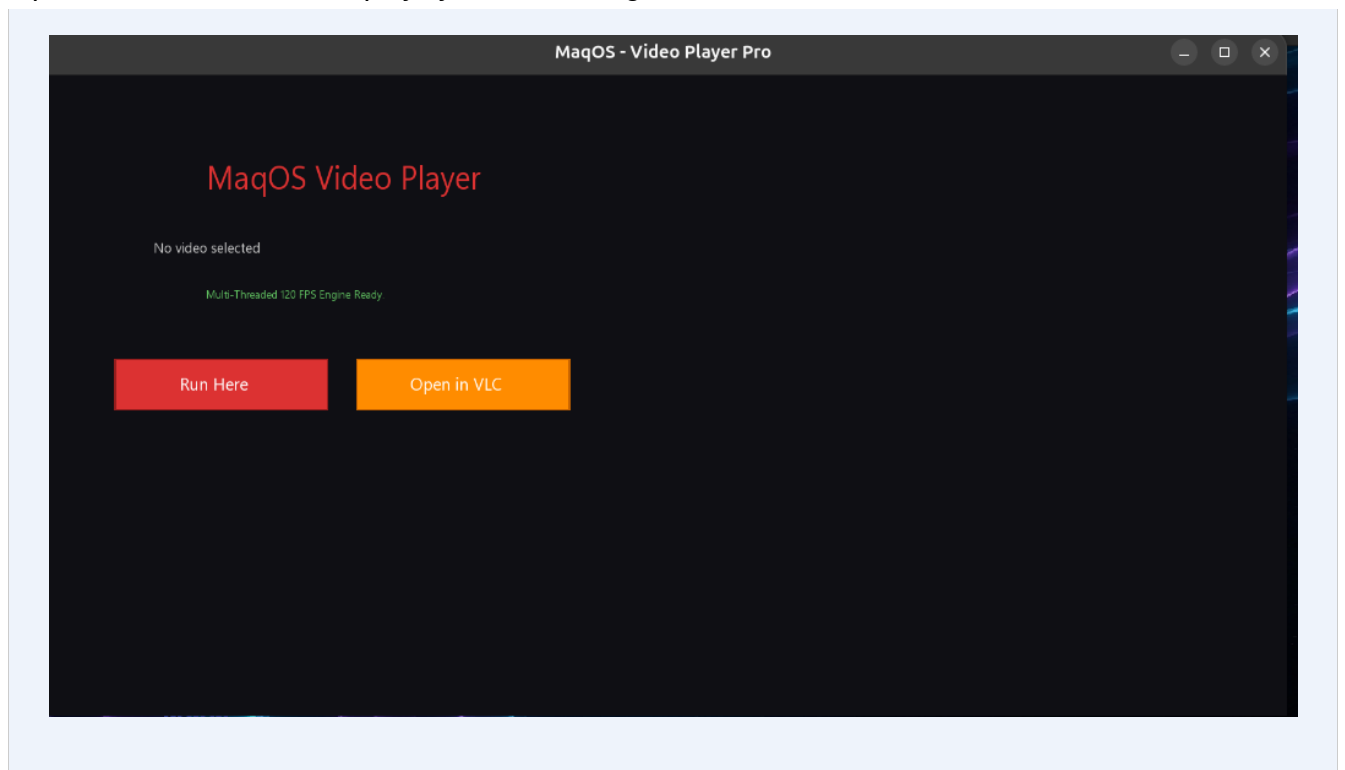
10.4.1 Audio Player

Launches audio playback via system audio infrastructure. SFML window provides playback controls.



10.4.2 Video Player

Opens video files via VLC/ffplay system calls. Highest resource cost: 300 MB RAM, 100 MB HDD.



10.5 Games

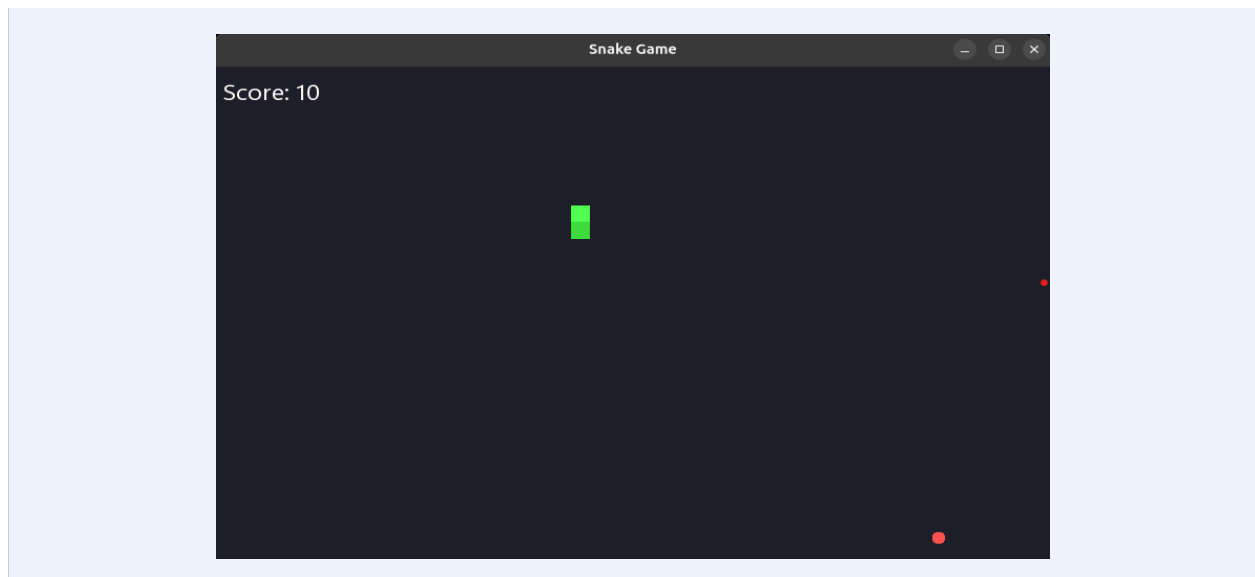
10.5.1 Chess

Two-player chess with rendered 8x8 board, piece move validation, turn management, and check detection.



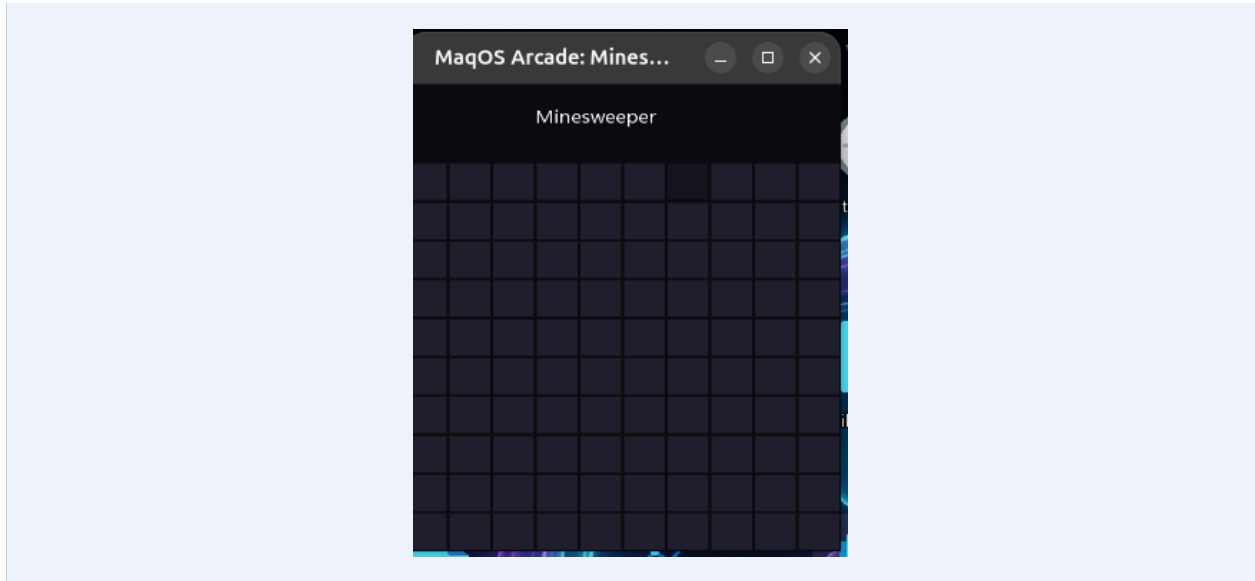
10.5.2 Snake

Classic snake with smooth 0.12s-per-move animation, score tracking, high-score persistence to file, and collision detection.



10.5.3 Minesweeper

Full Minesweeper: configurable grid, mine generation, recursive empty-cell reveal, flag placement, win/loss detection.



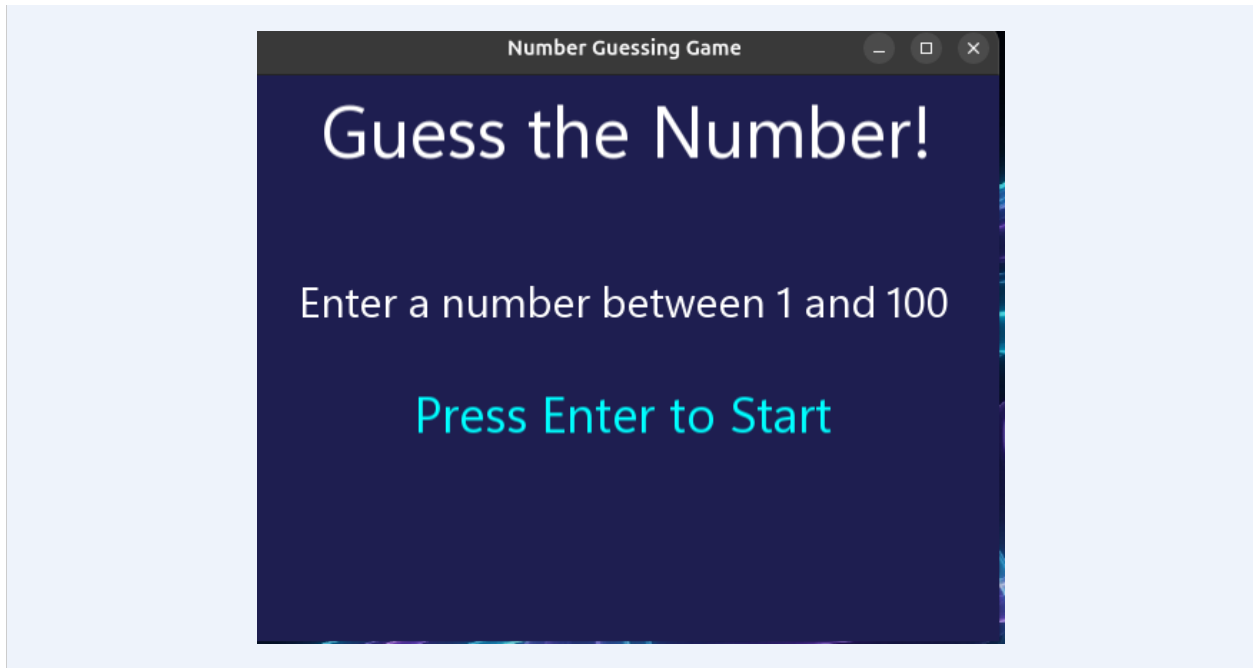
10.5.4 Tic Tac Toe

Two-player 3x3 grid with turn indicators, win-line highlighting, and restart button.



10.5.5 Number Guess

Number guessing game with configurable range, higher/lower hints, and attempt counter.



11. Inter-Process Communication

11.1 Pipe-Based Resource Handshake

Two anonymous pipes (req_pipe, ack_pipe) are created before each fork(). The child writes [ram_mb, hdd_mb] to req_pipe; the parent reads, decides, and writes ACK (1/0) to ack_pipe. All pipe ends are closed after use to prevent file descriptor leaks. This protocol ensures no child can consume resources not explicitly granted by the kernel.

11.2 Signal-Based Process Control

Signal	MaqOS Usage
SIGSTOP	Minimize process; simulate I/O block; Round Robin preemption
SIGCONT	Restore minimized process; resolve I/O interrupt; scheduler dispatch
SIGTERM	Close app (taskbar right-click); graceful shutdown sequence
SIGKILL	Kernel Mode force-kill from Kernel Panel (admin only)

11.3 System Event Log (maqos_system.log)

All OS events are appended to maqos_system.log with HH:MM:SS timestamps. A mutex (log_mutex) prevents interleaved writes. Logged events include:

- Process creation: PID, name, RAM and HDD allocated
- Resource grants and denials
- Scheduling: queue level, context switches, RR quantum expiry, Aging
- Deadlock detection results
- Mode switches
- Minimize, restore termination, I/O interrupt simulation and resolution events

```
System Log (maqos_system.log) □□□ Last 30 entries
[2026:53] CREATED: Minesweeper PID=4771 RAM=120MB
[2026:53] QUEUED (Level 1): Minesweeper PID=4771
[2026:53] CONTEXT SWITCH: Minesweeper -> Minesweeper
[2026:53] SCHEDULED (Level 1): Minesweeper PID=4771
[2026:55] RR QUANTUM EXPIRED: Minesweeper (simulated preemption)
[2027:41] TERMINATED: Minesweeper PID=4771
[2027:48] LAUNCH ATTEMPT: Number Guess exe= ./bin/numberGuess RAM=60 HDD=5
[2027:48] RAM GRANTED: 60MB, HDD: 5MB to Number Guess
[2027:48] CREATED: Number Guess PID=4781 RAM=60MB
[2027:48] QUEUED (Level 1): Number Guess PID=4781
[2027:48] CONTEXT SWITCH: Minesweeper -> Number Guess
[2027:48] SCHEDULED (Level 1): Number Guess PID=4781
[2027:50] RR QUANTUM EXPIRED: Number Guess (simulated preemption)
[2028:24] TERMINATED: Number Guess PID=4781
[2028:31] LAUNCH ATTEMPT: Task Manager exe= ./bin/task_manager RAM=120 HDD=5
[2028:31] RAM GRANTED: 120MB, HDD: 5MB to Task Manager
[2028:31] CREATED: Task Manager PID=4789 RAM=120MB
[2028:31] QUEUED (Level 1): Task Manager PID=4789
[2028:31] CONTEXT SWITCH: Number Guess -> Task Manager
[2028:31] SCHEDULED (Level 1): Task Manager PID=4789
[2028:33] RR QUANTUM EXPIRED: Task Manager (simulated preemption)
[2028:37] TERMINATED: Task Manager PID=4789
[2028:50] Switched to KERNEL MODE
[2028:50] LAUNCH ATTEMPT: KernelPanel exe= ./bin/K RAM=50 HDD=5
[2028:50] RAM GRANTED: 50MB, HDD: 5MB to KernelPanel
[2028:50] CREATED: KernelPanel PID=4798 RAM=50MB
[2028:50] CONTEXT SWITCH: Task Manager -> KernelPanel
[2028:50] SCHEDULED (Level 1): KernelPanel PID=4798
[2028:50] QUEUED (Level 1): KernelPanel PID=4798
[2028:52] RR QUANTUM EXPIRED: KernelPanel (simulated preemption)
```

12. How to Run MaqOS

12.1 Prerequisites (Ubuntu/Debian)

The Makefile installs all dependencies automatically on first build. Required packages:

- build-essential — GCC/G++ C++17 compiler
- libsfml-dev — SFML 2.x graphics, window, audio
- ffmpeg, vlc — media processing and video playback
- zenity — native file dialog for wallpaper selection
- pkg-config, libgtkmm-3.0-dev — additional build support

12.2 Build & Launch

```
unzip OS-main.zip && cd OS-main
make all                      # Build all + auto-install deps
./bin/startup                 # Recommended: full boot sequence
./bin/main 4 256 8            # Direct: 4GB RAM, 256GB HDD, 8 Cores
```

12.3 Runtime Controls

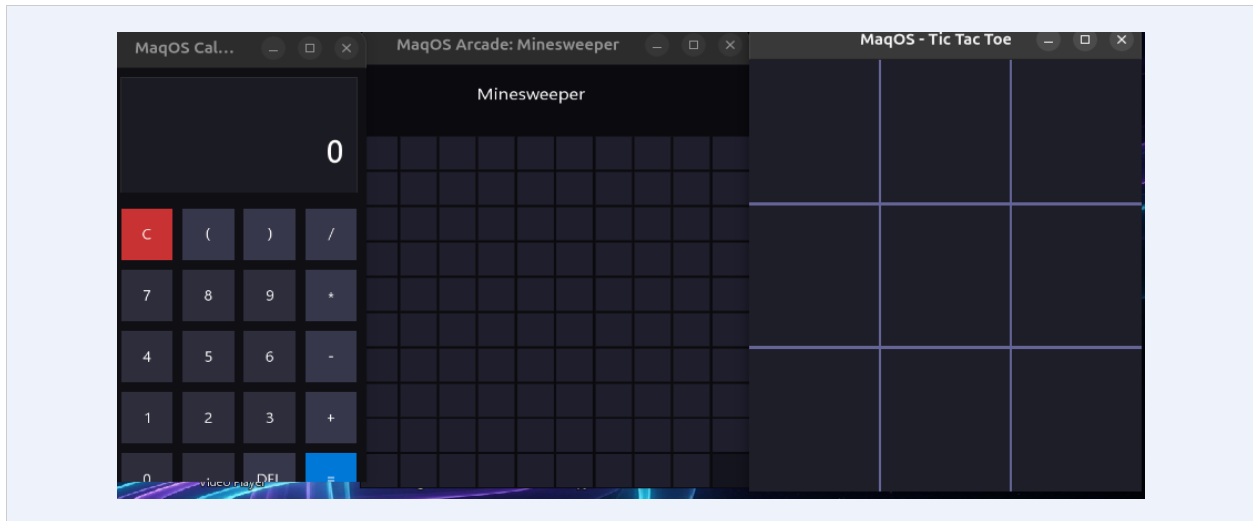
Input	Action
Left-click desktop icon	Launch app as new process (fork + exec)
Right-click desktop icon	Minimize (SIGSTOP) or restore (SIGCONT) running instance
Left-click taskbar app	Minimize or restore that process
Right-click taskbar app	Send SIGTERM — close the app
Start Button	Open / close Start Menu
Start Menu -> Power button	Initiate MaqOS shutdown sequence
Start Menu -> Set Wallpaper	Open Zenity file dialog to choose wallpaper
Kernel Mode button	Toggle User Mode <-> Kernel Mode
Key 'I'	Simulate I/O interrupt on first running process
Key 'F1'	Toggle on-screen instruction guide overlay
Window resize	Desktop auto-scales wallpaper and SFML view

13. OS Concepts Implemented

OS Concept	Implementation in MaqOS
Process Creation	fork() + exec() with IPC pipe handshake for RAM/HDD negotiation
Process States	READY, RUNNING, MINIMIZED, BLOCKED (I/O), TERMINATED
CPU Scheduling	3-level MLQ: FCFS (System), Priority+RR (Interactive), FCFS (Background)
Round Robin	2-second quantum timer thread for Q1 interactive processes
Priority Scheduling	std::priority_queue (min-heap) for Q1 ordered by ProcessEntry.priority
Aging	wait_ticks counter; every 10 ticks -> priority decremented by 1
Context Switching	context_switch() logs transitions; SIGSTOP/SIGCONT for preemption
Semaphores	sem_t ram_sem (binary), sem_t core_sem (counting) — POSIX semaphore.h
Mutex / Critical Sections	std::mutex for process table, queues, log file, RAM counter
Memory Management	RAM + HDD pools; per-process allocation tracked in ProcessEntry
Deadlock Detection	Resource Allocation Graph + DFS cycle detection; runs every 5 seconds
IPC — Pipes	Anonymous pipes for parent-child resource request/ACK handshake
IPC — Signals	SIGSTOP, SIGCONT, SIGTERM, SIGKILL for full process lifecycle control
Kernel / User Mode	Privilege toggle; Kernel Mode auto-launches Kernel Panel with admin access
Interrupt Simulation	Key 'I' triggers BLOCKED state; 3-second auto-resolution via detached thread
Child Reaping	waitpid(WNOHANG) in main loop prevents zombie processes
Multithreading	pthreads for scheduler, deadlock monitor, and Notepad auto-save
File System Ops	Create, read, delete, copy, move — full POSIX file API
System Logging	Timestamped maqos_system.log capturing all kernel-level events

14. Additional Screenshots

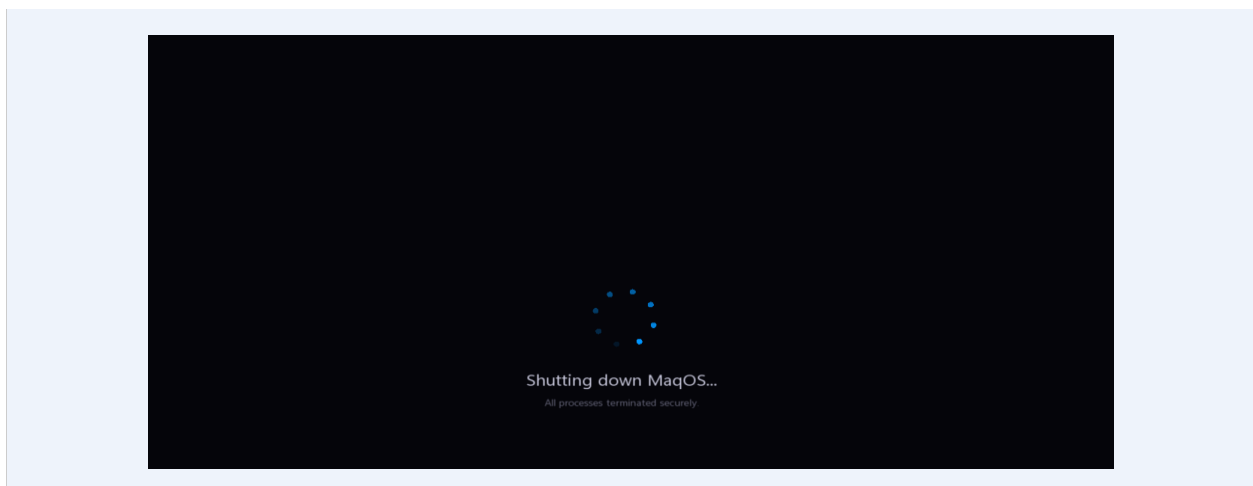
14.1 Multiple Apps Running Simultaneously



14.2 Scheduling & Context Switch Log

```
[20:33:41] CONTEXT SWITCH: Tic Tac Toc -> Calculator
[20:33:41] SCHEDULED (Level 1): Calculator PID=4885
[20:33:41] QUEUED (Level 1): Calculator PID=4885
[20:33:43] RR QUANTUM EXPIRED: Calculator (simulated preemption)
[20:34:07] LAUNCH ATTEMPT: Minesweeper exe= /bin/minesweeper RAM=120 HDD=10
[20:34:07] RAM GRANTED: 120MB, HDD: 10MB to Minesweeper
[20:34:07] CREATED: Minesweeper PID=4894 RAM=120MB
[20:34:07] CONTEXT SWITCH: Calculator -> Minesweeper
[20:34:07] SCHEDULED (Level 1): Minesweeper PID=4894
[20:34:07] QUEUED (Level 1): Minesweeper PID=4894
[20:34:09] RR QUANTUM EXPIRED: Minesweeper (simulated preemption)
[20:35:30] TERMINATED: Calculator PID=4885
[20:35:33] TERMINATED: Minesweeper PID=4894
[20:35:34] TERMINATED: Tic Tac Toc PID=4877
[20:35:35] LAUNCH ATTEMPT: Task Manager exe= /bin/task_manager RAM=120 HDD=5
```

14.5 Shutdown Screen



15. Conclusion

MaqOS (Maqsad Operating System) successfully demonstrates all core operating system principles within an engaging, graphically rich simulation environment. By leveraging Linux system calls directly — `fork()`, `exec()`, `waitpid()`, POSIX signals, anonymous pipes, and POSIX semaphores — the project goes beyond simulation to deliver genuine multi-process behavior with real OS-level process management.

The three-level multilevel queue scheduler with aging, the Resource Allocation Graph deadlock detection engine, the semaphore-protected RAM/HDD resource pools, and the User/Kernel mode privilege system collectively cover all major concepts from the CL-2006 Operating Systems Lab curriculum. The five flowcharts and architecture diagrams (Figures 1-5) provide clear visual documentation of every subsystem design decision.

The SFML-based desktop makes abstract OS concepts tangible and interactive — users can observe process states changing in real time in the Task Manager, read scheduling decisions in `maqos_system.log`, trigger I/O interrupts with a keypress, and experience resource denial when the simulated RAM pool is exhausted.

15.1 Key Achievements

- 19 fully functional applications running as independent OS processes via `fork()` + `exec()`.
- IPC pipe handshake ensuring atomic resource negotiation before any app launches.
- 3-level MLQ scheduler (System/Interactive/Background) with Round Robin + aging.
- POSIX semaphore-based RAM and core management with real-time GUI dashboard.
- Resource Allocation Graph deadlock detection running as a background daemon thread.
- Signal-based process control (SIGSTOP/SIGCONT/SIGTERM) for minimize/restore/close.
- Comprehensive timestamped system event log capturing all kernel-level events.
- Polished Windows 11-inspired SFML GUI with glassmorphism taskbar and Start Menu.
- Privileged Kernel Mode with admin panel for elevated process management.